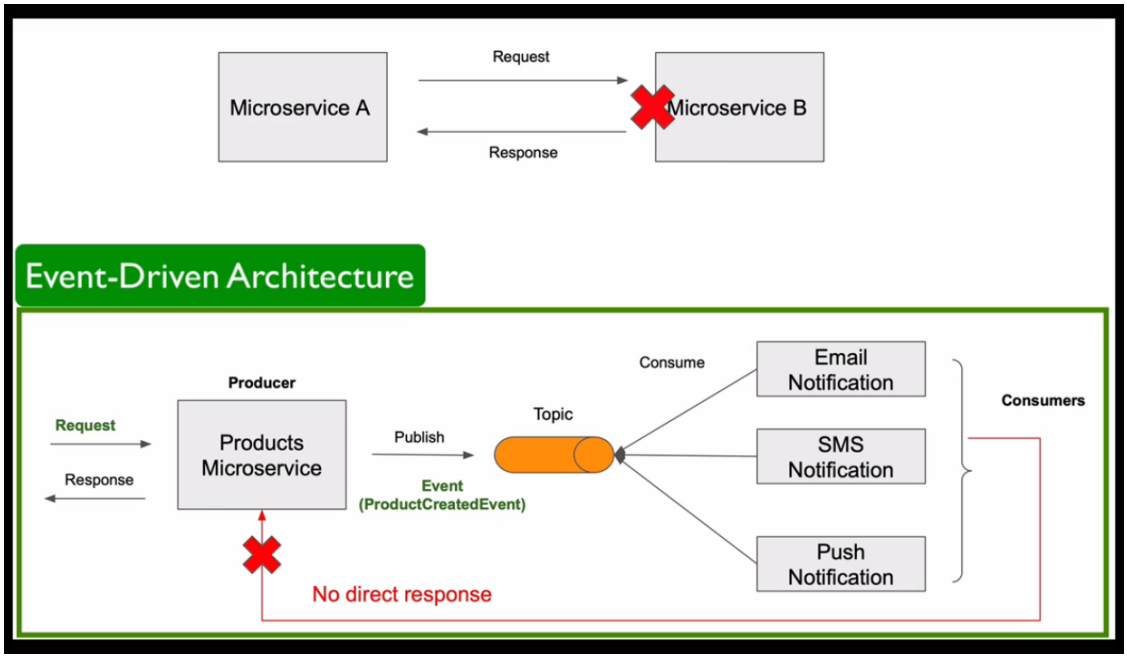
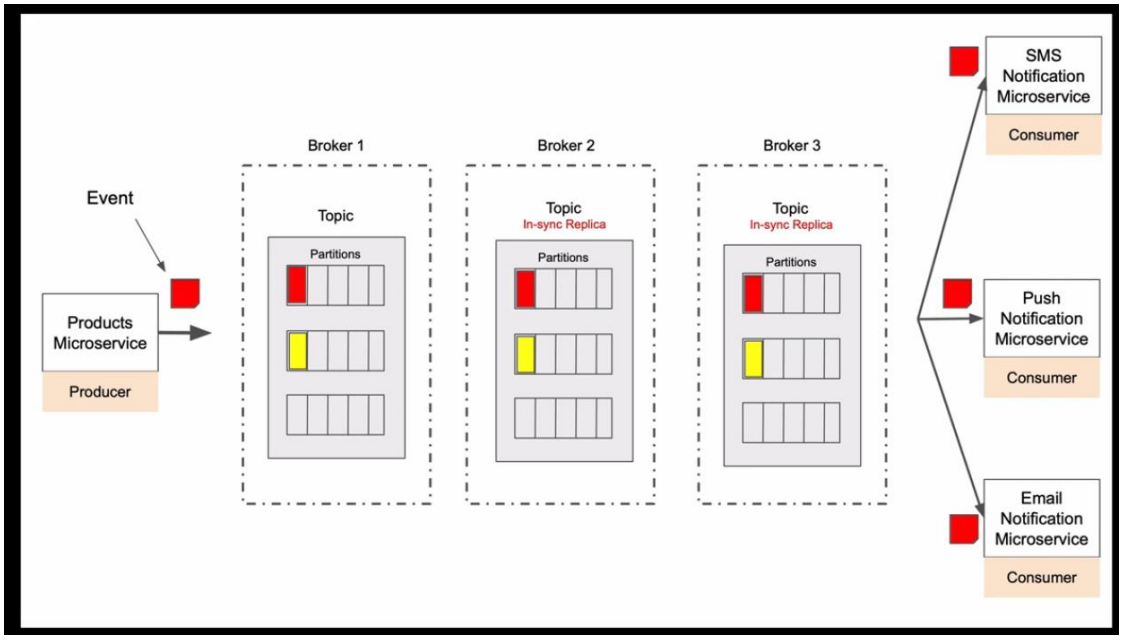




Kafka assigns:

```
Partition 0 -> Consumer 1
Partition 1 -> Consumer 2
Partition 2 -> Consumer 3
```

Each partition is consumed by **only one consumer within a consumer group**.

This avoids duplicate processing.

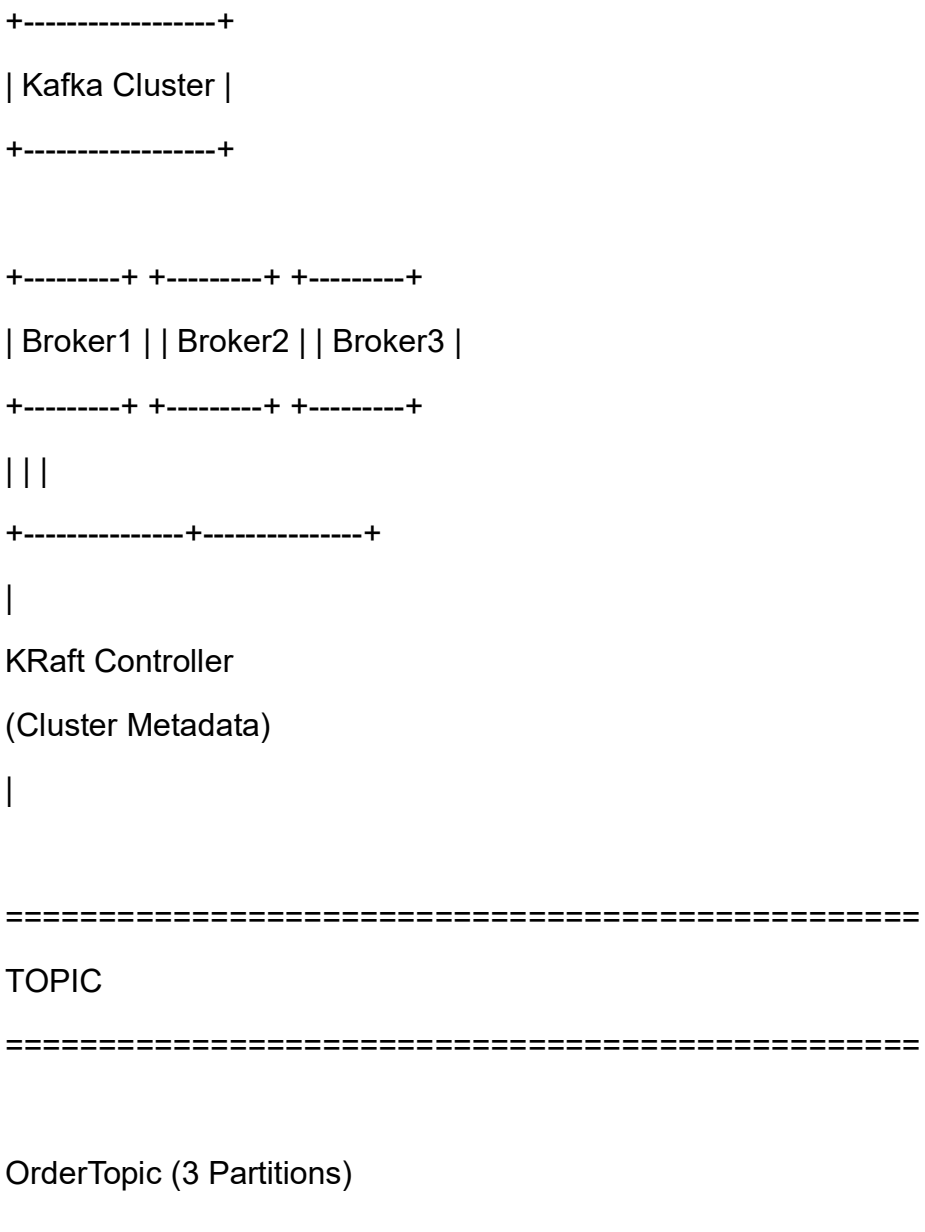
Kafka Architecture

Concept	What It Is	Example
Cluster	Collection of Kafka brokers	3 Kafka servers together
Broker	One Kafka server instance	Broker-1
Topic	Logical category of messages	`orders`
Partition	Physical log storing messages	Partition-0
Replica	Copy of a partition	Follower copy
Leader Replica	Handles reads/writes	Partition-0 on Broker-1
Follower Replica	Copies data from leader	Same partition on Broker-2

Hierarchy

Plain Text

```
Kafka Cluster
    |
    +-- Broker 1
    +-- Broker 2
    +-- Broker 3
            |
            +-- Topic: Orders
                    |
                    +-- Partition 0
                    +-- Partition 1
                    +-- Partition 2
```


+-----------------+
| Kafka Cluster |
+-----------------+
+---------+ +---------+ +---------+
| Broker1 | | Broker2 | | Broker3 |
+---------+ +---------+ +---------+
| | |
+--------------+--------------+
|
KRaft Controller
(Cluster Metadata)
|
==========
TOPIC
==========
OrderTopic (3 Partitions)
P0 P1 P2
------- ------- -------
Leader Leader Leader
B1 B2 B3
Replica Replica Replica
B2 B3 B1
Replica Replica Replica

B3 B1 B2

==========

Producers ---> Kafka ---> Consumers

Example: 3 Brokers, Replication Factor = 3

Plain Text

```
Partition 0
Leader   -> Broker 1
Follower -> Broker 2
Follower -> Broker 3

Partition 1
Leader   -> Broker 2
Follower -> Broker 3
Follower -> Broker 1

Partition 2
Leader   -> Broker 3
Follower -> Broker 1
Follower -> Broker 2
```

Consumer Group

Multiple instances of the same application.

```
order-service

Consumer 1
Consumer 2
Consumer 3

```

All belong to the same consumer group.

Partition Assignment

```
Partitions: 3

P0 -> Consumer 1
P1 -> Consumer 2
P2 -> Consumer 3
```

Rule:

```
1 Partition
    ->
1 Consumer in a group
```

More Consumers than Partitions

```
Partitions = 3

Consumers:
C1
C2
C3
C4
```

Result:

```
P0 -> C1
P1 -> C2
P2 -> C3

C4 -> Idle
```

Producer ACKs

These control how safely messages are written to Kafka.

Setting	What Happens	Speed	Reliability
`acks=0`	Producer doesn't wait for any response	Fastest	Lowest
`acks=1`	Leader acknowledges	Fast	Medium
`acks=all`	Leader waits for replicas	Slowest	Highest

acks=0

```
Producer -> Broker

No confirmation
```

Risk:

```
Message may be lost
```

acks=1

```
Producer
    |
Leader Broker
    |
ACK
```

Risk:

```
Leader crashes before replication

```

Possible loss.

acks=all

```
Producer
    |
Leader
    |
Followers replicated
    |
ACK
```

Safest option.

Delivery Guarantees

At Most Once

```
Read
Commit
Process
```

Risk:

```
Message loss
```

No duplicates.

At Least Once (Most Common)

```
Read
Process
Commit
```

Risk:

```
Duplicates possible
```

No message loss.

acks = "Was the message safely written?"
delivery guarantee = "How many times was the message delivered?"

The ACK itself comes from Kafka to the producer, and the producer uses that ACK (or lack of ACK) to decide whether to retry, which determines the delivery guarantee.

Exactly Once

```
Process exactly one time
```

Implemented using:

```
Transactions
Idempotent Producer
```

More complex.

Yes, **till now we're only talking about Producer → Kafka Broker**.

```
Producer ---> Kafka Broker
          (acks, retries,
           idempotence,
           delivery guarantees)
```

Things we've discussed:

- `acks`
- `retries`
- `enable.idempotence`
- `max.in.flight.requests.per.connection`
- At-most-once
- At-least-once
- Exactly-once (producer side)

The **consumer** comes into the picture with a different concern:

```
Producer ---> Kafka ---> Consumer

```

Consumer worries about:

- Offsets
- Offset commits
- Auto commit vs manual commit
- Reprocessing messages
- Consumer retries
- DLT
- Consumer-side delivery guarantees

✅ There is **no global offset = 3**.

Kafka tracks offsets **separately for every partition within a consumer group**.

1. How does Kafka store consumer offsets?

Kafka stores consumer offsets in an internal Kafka topic called:

```
__consumer_offsets
```

Kafka writes records like:

```
Consumer Group: order-service
Partition: P0
Committed Offset: 150
```

So Kafka itself stores and replicates offset information.

2. What happens if a consumer restarts?

When a consumer restarts:

1. Joins the consumer group.
2. Reads the last committed offset from `__consumer_offsets`.
3. Continues consuming from there.

Example:

```
Last committed offset = 100
```

After restart:

```
Start reading from Offset 101
```

5. Can offsets be reset manually?

✅ Yes.

You can move offsets:

To earliest

```
kafka-consumer-groups.sh \
--reset-offsets \
--to-earliest
```

Consumer starts from beginning.

To latest

```
--to-latest
```

Consumer skips old messages and starts from newest.

6. What happens if offset is lost?

Suppose Kafka cannot find the committed offset.

Kafka checks:

```
auto.offset.reset

```

earliest

```properties
auto.offset.reset=earliest
```

Read from beginning.

```
0,1,2,3,...
```

Examples:

```properties
max.poll.records=1
```

Read one message at a time.

```properties
max.poll.records=500

```

Read up to 500 messages per poll.

Consumer Lag

The difference between:

```
Latest Offset
-
Consumer Committed Offset
```

is called **Consumer Lag**.

Example:

```
Latest Offset = 10000
Consumer Offset = 7000
```

Lag:

```
3000 messages
```

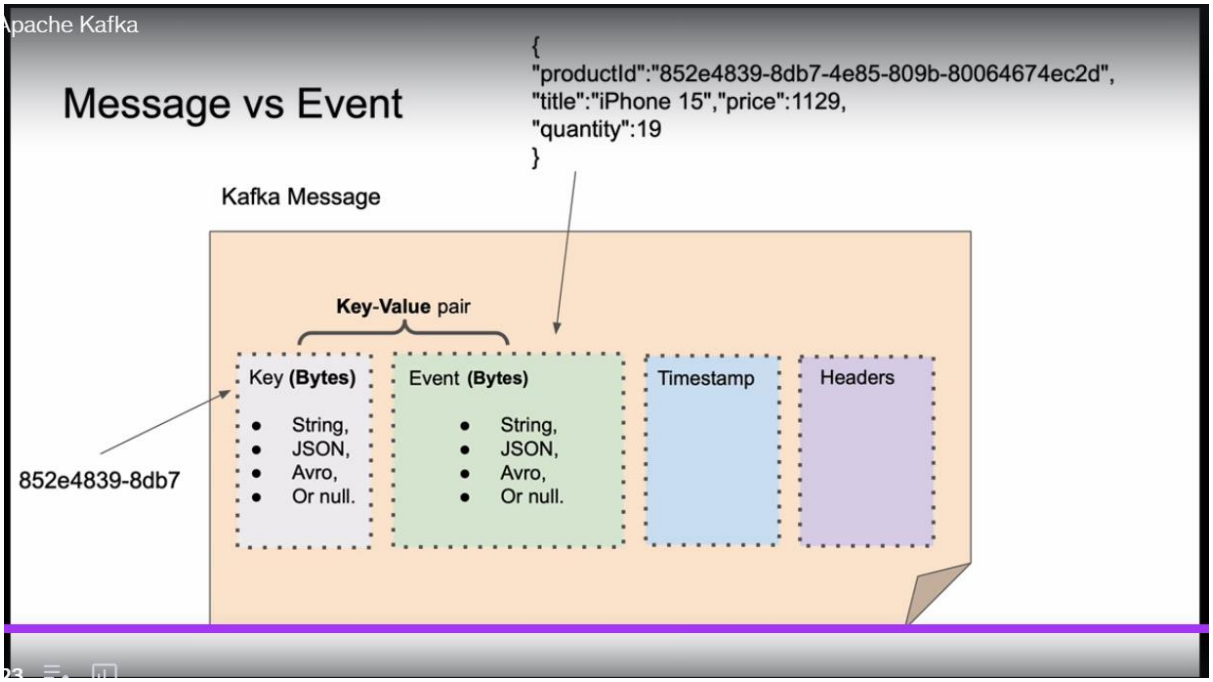
Meaning consumer is behind by 3000 record

How do we handle slow consumers?

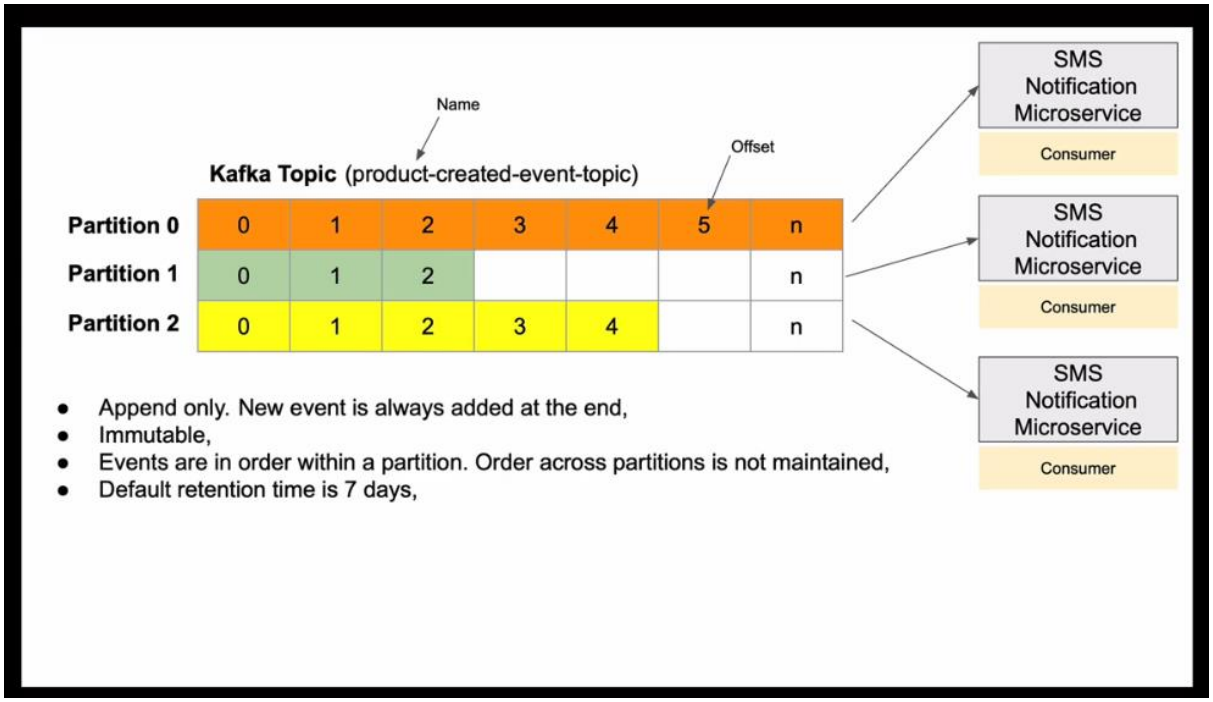
1. Add more consumers

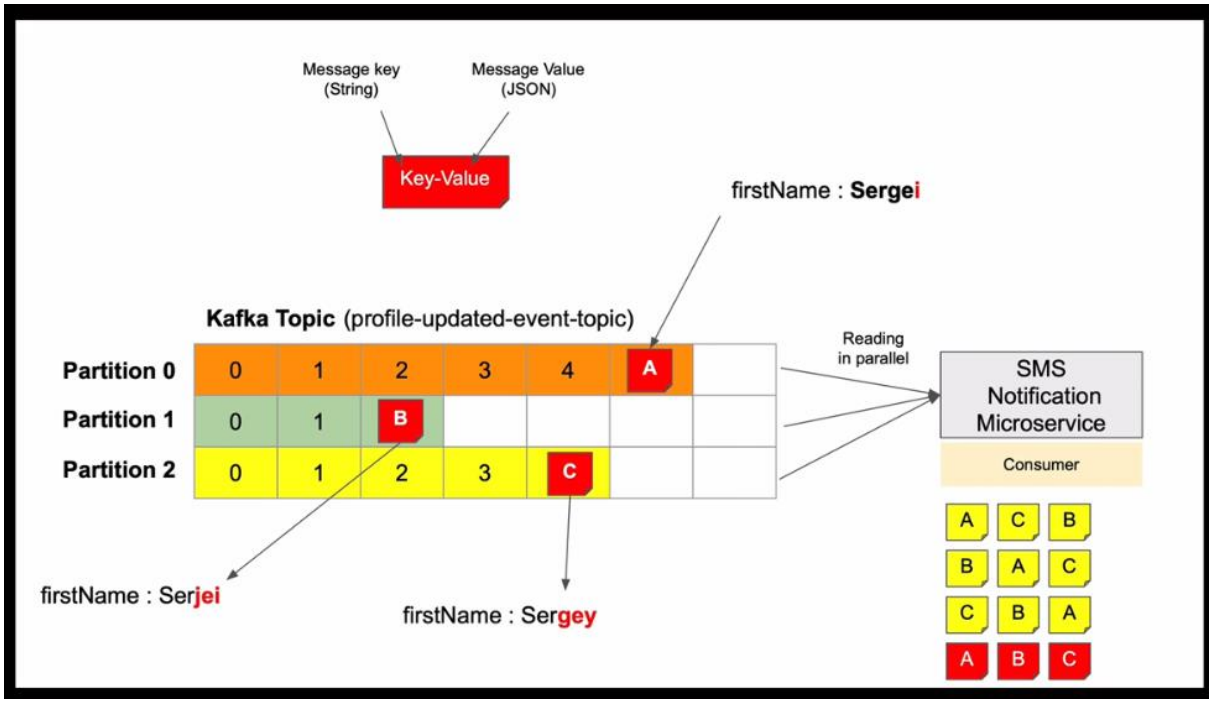
2. Increase partitions

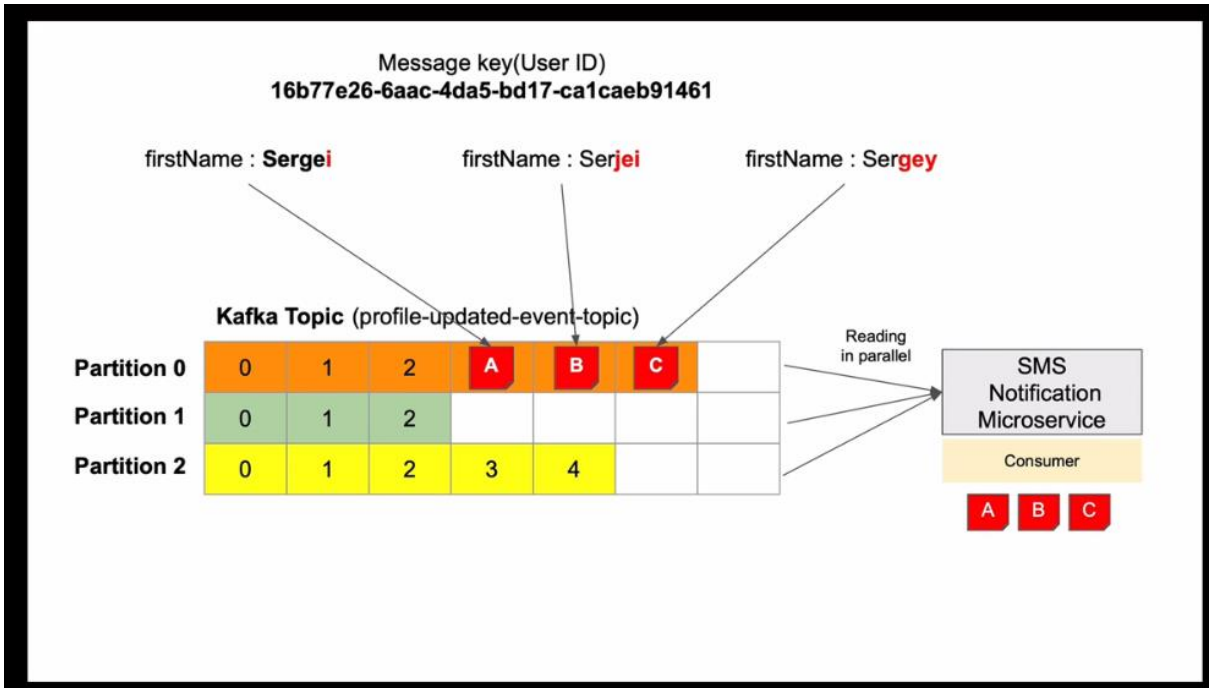
3. Optimize consumer processing



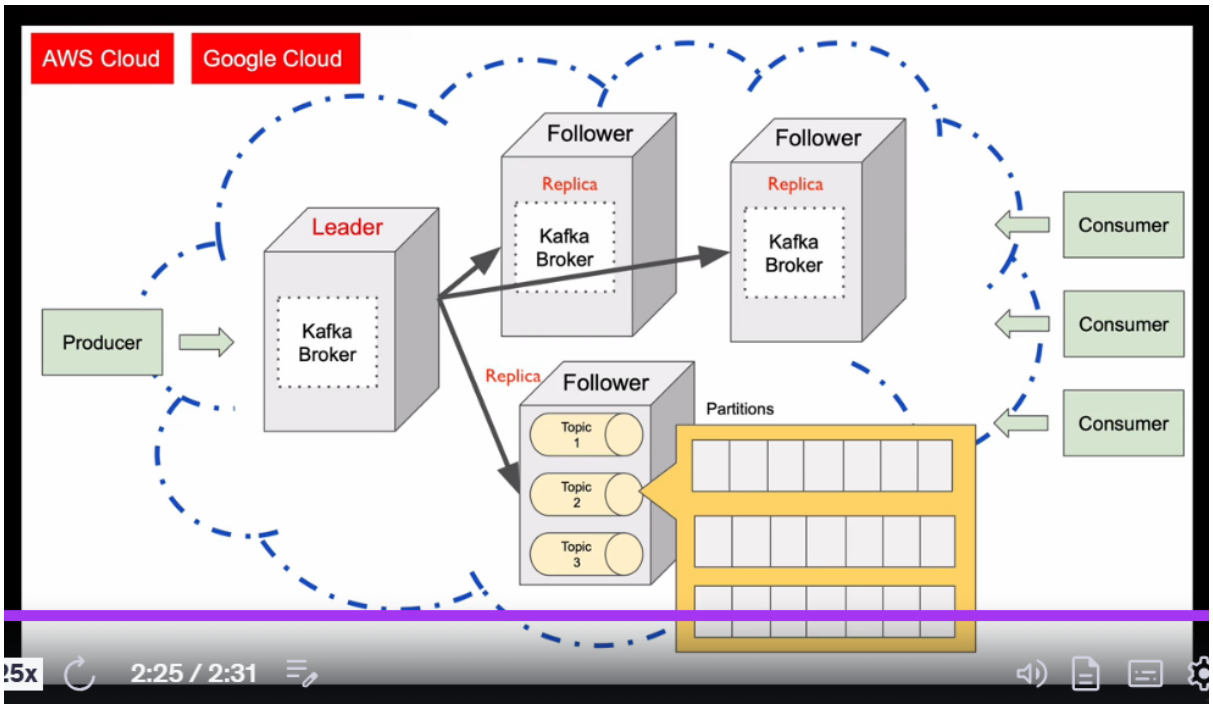
An event represents something that happened in the business, such as "Order Created". A Kafka message is the actual data record carrying that event. Topics are logical channels used to group related events, such as orders, payments, or customers. Producers publish event messages to topics, and consumers subscribe to those topics to process the events.




Message key
(String)
Message Value
(JSON)
Key-Value
firstName : Sergei
Kafka Topic (profile-updated-event-topic)
Partition 0
Partition 1
Partition 2
Reading
in parallel
SMS
Notification
Microservice
Consumer
firstName : Serjei
firstName : Sergey


Message key(User ID)
16b77e26-6aac-4da5-bd17-ca1caeb91461
firstName : Sergei
firstName : Serjei
firstName : Sergey
Kafka Topic (profile-updated-event-topic)
Partition 0
Partition 1
Partition 2
Reading
in parallel
SMS
Notification
Microservice
Consumer

KAFKA BROKER:



Distribution across brokers:

```
Broker1:
  P0 (Leader)
  P1 (Follower)
  P2 (Follower)

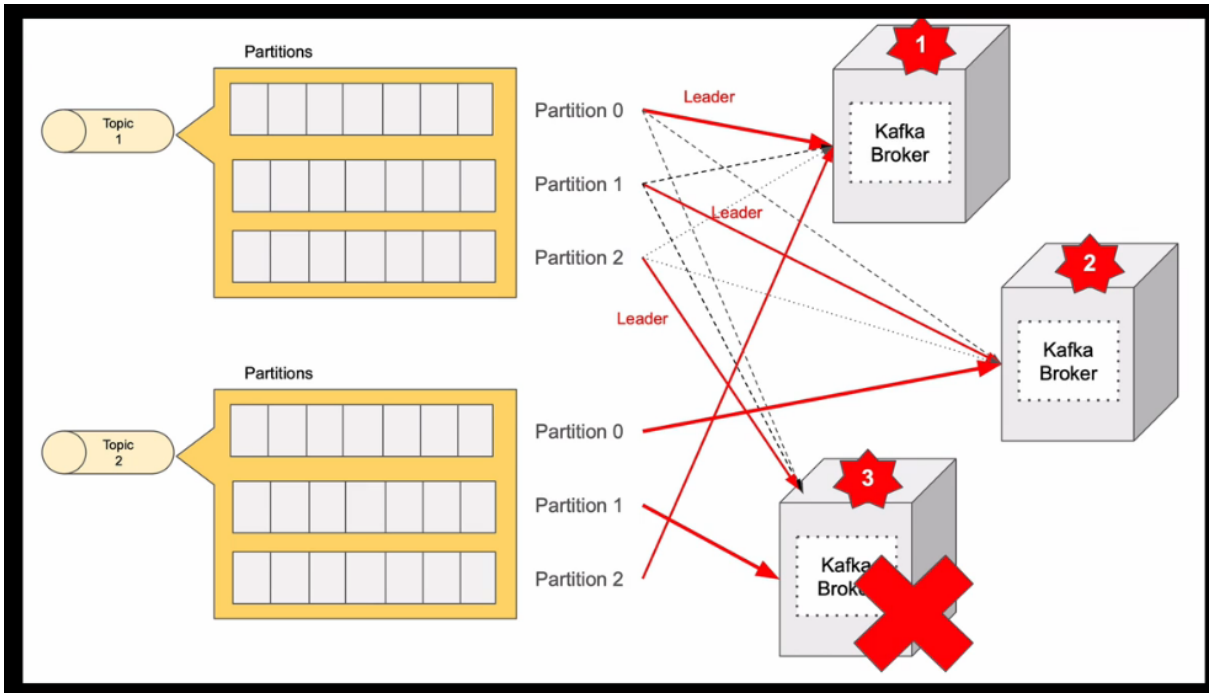
Broker2:
  P0 (Follower)
  P1 (Leader)
  P2 (Follower)

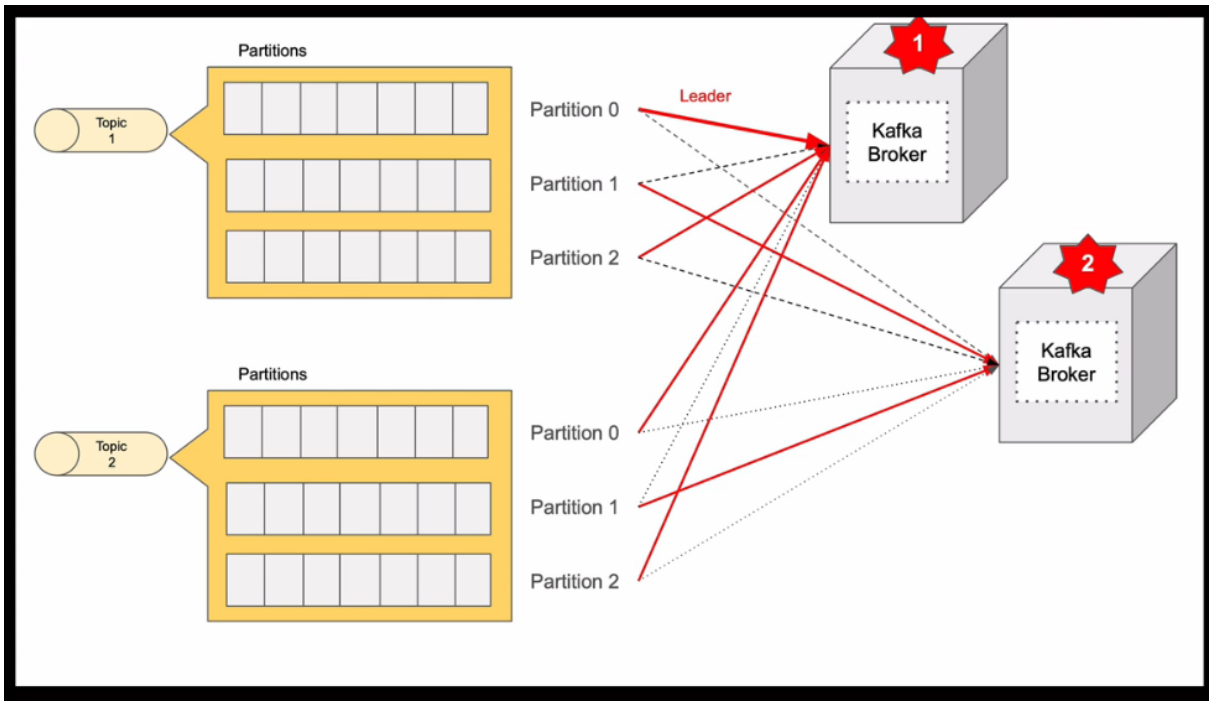
Broker3:
  P0 (Follower)
  P1 (Follower)
  P2 (Leader)
```

So:

- Broker1 is leader for P0
- Broker2 is leader for P1
- Broker3 is leader for P2

All three brokers are actively working.


Partitions
Topic 1
Partition 0
Partition 1
Partition 2
Leader
Leader
Leader
1
Kafka Broker
2
Kafka Broker
Partitions
Topic 2
Partition 0
Partition 1
Partition 2
3
Kafka Brok


Partitions
Topic 1
Partition 0
Partition 1
Partition 2
Leader
1
Kafka Broker
2
Kafka Broker
Partitions
Topic 2
Partition 0
Partition 1
Partition 2

✓ What you know ⓘ

In the context of Apache Kafka, what is a Broker?

In the context of Apache Kafka, what is a Consumer?

In the context of Apache Kafka, what is a Producer?

You can run multiple Apache Kafka servers in a cluster. True or False?

What is Apache Kafka primarily used for?

✓ What you know ⓘ

How does Kafka ensure message ordering within a partition?

Can a single Kafka Topic have multiple Partitions?

How does Kafka ensure message ordering in a Topic Partition?

What does a Partition within a Kafka Topic represent?

What is a Kafka Topic primarily used for in the Kafka architecture?

You can increase the number of partitions in a Kafka topic to scale. But once set, partiti

✓ What you know ⓘ

What is the role of a Follower broker in a Kafka cluster?

What happens when a leader broker for a partition becomes unavailable?

What is the role of a Leader broker in a Kafka cluster?

What happens when a Kafka broker goes down?

How does Kafka ensure message durability?

What is the primary role of a broker in Apache Kafka?

Think of it this way:

```
Kafka Brokers = Store and serve messages
ZooKeeper/KRaft = Manage the Kafka cluster
```

What do they do?

1. Broker Registration

When a broker starts:

```
Broker1 starts
Broker2 starts
Broker3 starts
```

ZooKeeper-Based Kafka (Old Architecture)

```
            ZooKeeper
               |
     ----------------------
     |         |          |
Broker1    Broker2    Broker3
```

ZooKeeper maintained cluster state and coordinated leader elections.

Drawbacks

- Extra component to manage
- More operational complexity
- Separate cluster to maintain
- Additional network communication

KRaft (Kafka Raft)

New Kafka versions remove ZooKeeper.

```
      Kafka Controllers
             |
    --------------------
    |        |         |
Broker1 Broker2 Broker3
```

Metadata is stored inside Kafka itself using the **Raft consensus protocol**.

What does a KRaft Controller do?

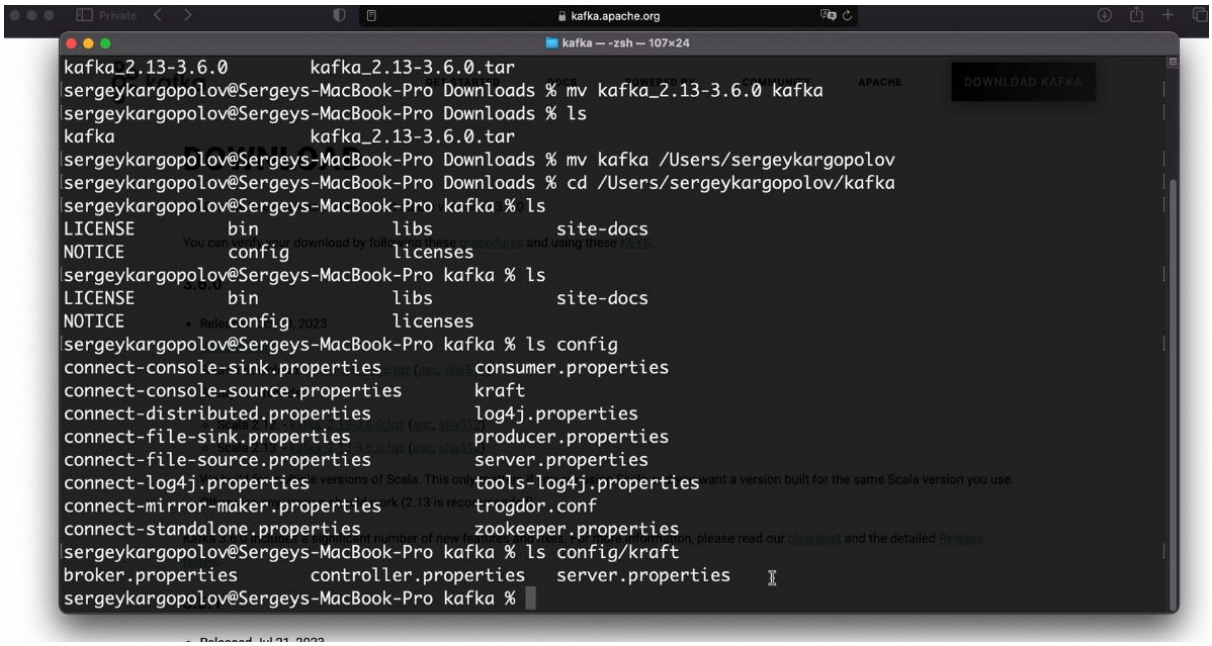
The KRaft controller is responsible for:

1. Metadata Management

Stores information about:

- Topics
- Partitions
- Replicas
- Brokers
- ACLs
- Configurations

Instead of ZooKeeper storing this metadata, KRaft controllers store and replicate it. apache +1



```
sergeykargopolov@Sergeys-MacBook-Pro kafka % ls bin/windows
connect-distributed.bat                 kafka-leader-election.bat
connect-plugin-path.bat                 kafka-log-dirs.bat
connect-standalone.bat                  kafka-metadata-quorum.bat
kafka-acls.bat                          kafka-mirror-maker.bat
kafka-broker-api-versions.bat           kafka-producer-perf-test.bat
kafka-cluster.bat                       kafka-reassign-partitions.bat
kafka-configs.bat                       kafka-replica-verification.bat
kafka-console-consumer.bat              kafka-run-class.bat
kafka-console-producer.bat              kafka-server-start.bat
kafka-consumer-groups.bat               kafka-server-stop.bat
kafka-consumer-perf-test.bat            kafka-storage.bat
kafka-delegation-tokens.bat             kafka-streams-application-reset.bat
kafka-delete-records.bat                kafka-topics.bat
kafka-dump-log.bat                      kafka-transactions.bat
kafka-e2e-latency.bat                   zookeeper-server-start.bat
kafka-features.bat                      zookeeper-server-stop.bat
kafka-get-offsets.bat                   zookeeper-shell.bat
kafka-jmx.bat
sergeykargopolov@Sergeys-MacBook-Pro kafka %
```

```
sergeykargopolov@Sergeys-MacBook-Pro kafka % ./bin/kafka-storage.sh random-uuid
4hxUhnt4SKCCCTC6br5Cgg
sergeykargopolov@Sergeys-MacBook-Pro kafka % ./bin/kafka-storage.sh format -t 4hxUhnt4SKCCCTC6br5Cgg -c con
fig/kraft/server.properties
Formatting /tmp/kraft-combined-logs with metadata.version 3.6-IV2.
sergeykargopolov@Sergeys-MacBook-Pro kafka % ./bin/kafka-server-start.sh config/kraft/server.properties
```

For **KRaft mode**, you just start Kafka directly (no ZooKeeper):

```shell
bin/kafka-server-start.sh config/kraft/server.properties
```

If you're on Windows:

```bat
bin\windows\kafka-server-start.bat config\kraft\server.properties
```

To stop the server:

```shell
bin/kafka-server-stop.sh
```

Typical flow:

```shell
# Start Kafka (KRaft)
bin/kafka-server-start.sh config/kraft/server.properties

# Create topic
bin/kafka-topics.sh --create \
  --topic test \
  --bootstrap-server localhost:9092 \
  --partitions 3 \
  --replication-factor 1
```

Running Multiple Kafka Broker

Yes. For a **multi-broker KRaft cluster**, you typically create **multiple broker config files**, one per broker.

For example:

```
config/kraft/server1.properties
config/kraft/server2.properties
config/kraft/server3.properties
```

Each file has a different:

```properties
node.id=1
listeners=PLAINTEXT://:9092
log.dirs=/tmp/kraft-broker-1
```

```properties
node.id=2
listeners=PLAINTEXT://:9093
log.dirs=/tmp/kraft-broker-2
```

```properties
node.id=3
listeners=PLAINTEXT://:9094
log.dirs=/tmp/kraft-broker-3
```

Then start each broker separately:

Then start each broker separately:

```shell
bin/kafka-server-start.sh config/kraft/server1.properties
```

```shell
bin/kafka-server-start.sh config/kraft/server2.properties
```

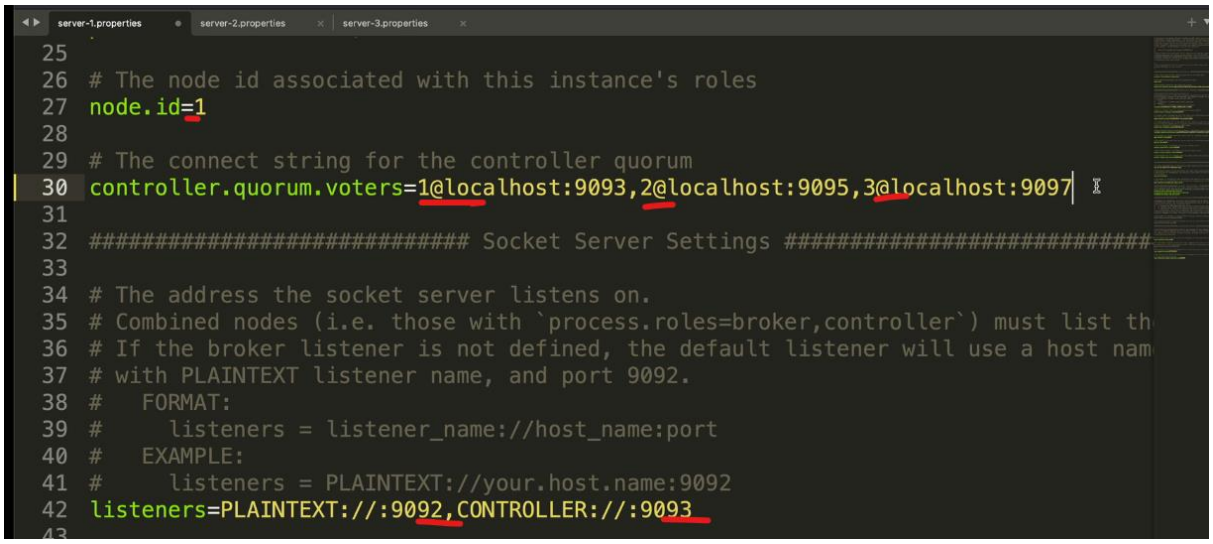
```shell
bin/kafka-server-start.sh config/kraft/server3.properties
```

So the practical answer is:

> Yes, you usually copy `server.properties` and create 3 versions with different `node.id`, `listeners` (ports), and `log.dirs`. Then start three Kafka broker processes, one using each config file.

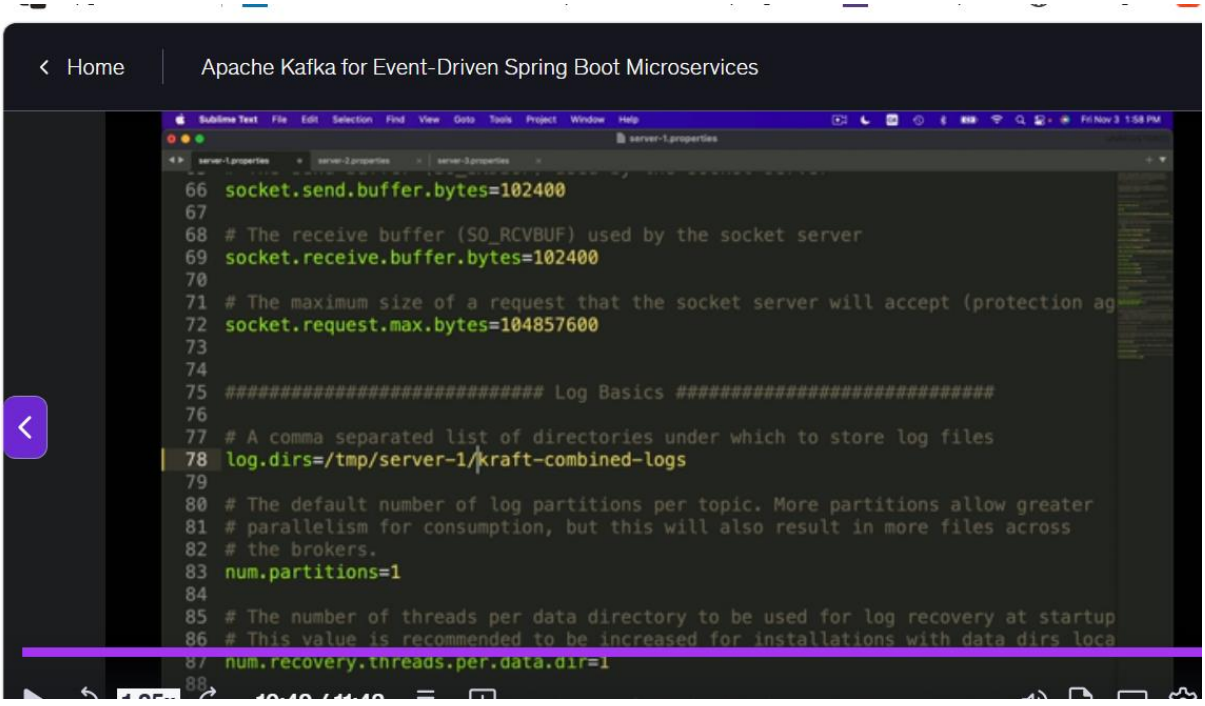
This can be on:

- **3 different machines** (common in production), or
- **1 machine with different ports** (common for learning/testing).

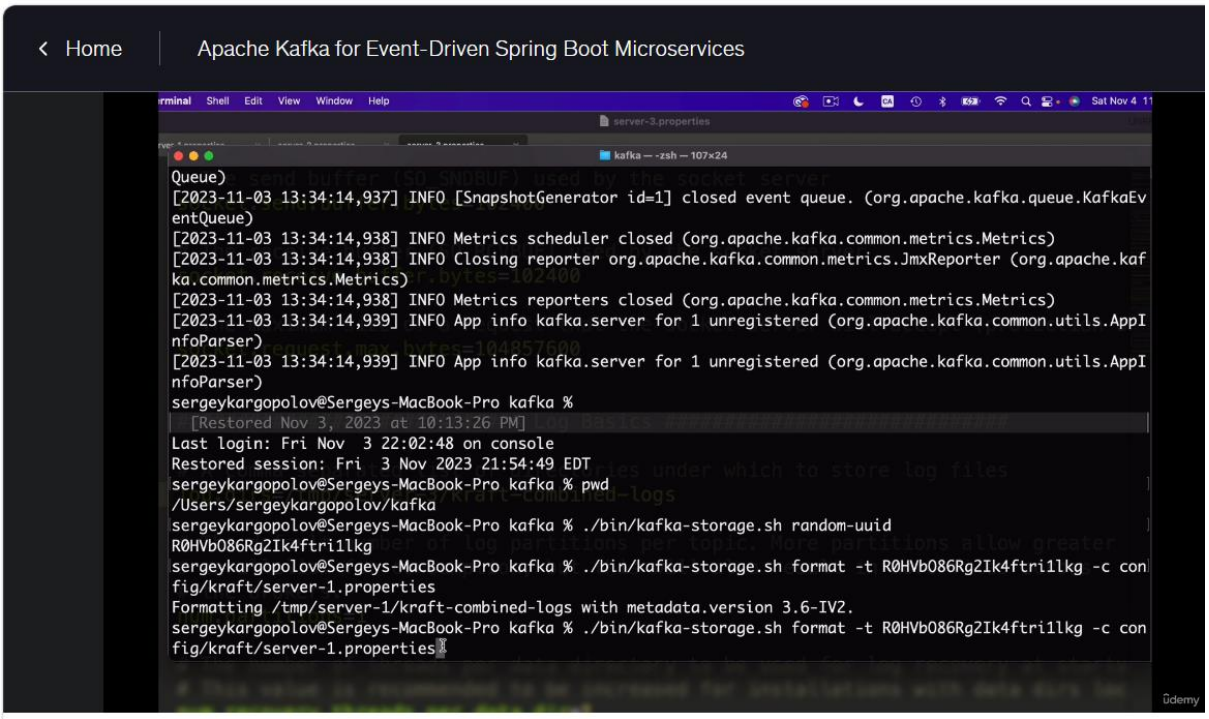


```
37 # with PLAINTEXT listener name, and port 9092.
38 #   FORMAT:
39 #     listeners = listener_name://host_name:port
40 #   EXAMPLE:
41 #     listeners = PLAINTEXT://your.host.name:9092
42 listeners=PLAINTEXT://:9094,CONTROLLER://:9095
43
44 # Name of listener used for communication between brokers.
45 inter.broker.listener.name=PLAINTEXT
46
47 # Listener name, hostname and port the broker will advertise to cl
48 # If not set, it uses the value for "listeners".
49 advertised.listeners=PLAINTEXT://localhost:9094
50
51 # A comma-separated list of the names of the listeners used by the
52 # If no explicit mapping set in `listener.security.protocol.map`,
53 # This is required if running in KRaft mode.
54 controller.listener.names=CONTROLLER
55
56 # Maps listener names to security protocols, the default is for th
57 listener.security.protocol.map=CONTROLLER:PLAINTEXT,PLAINTEXT:PLAI
58
59 # The number of threads that the server uses for receiving request
```

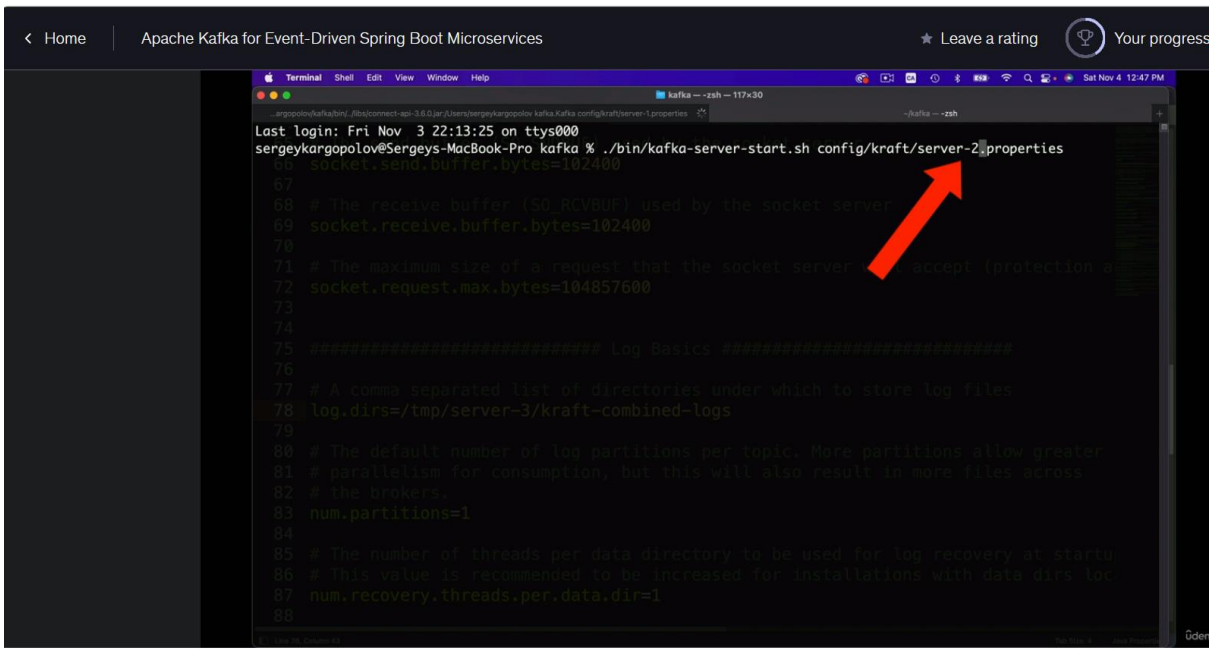
Log directories: Different servers / brokers different log files



Run for same cluster id for the three server



Starts all the server



1. Download Kafka

2. Generate Cluster UUID

3. Create server1.properties

4. Create server2.properties

5. Create server3.properties

6. Run kafka-storage.sh format for all 3 brokers

7. Start Broker 1

CREATE TOPIC:
./kafka-topics.sh –create –topic sample –partition 3 –replication-factor 3 –bootstrap-server localhost:9092, localhost:9093



What happens?

`--bootstrap-server` is **just the broker Kafka initially connects to.**

Kafka connects to Broker1, then Broker1 returns the metadata of the entire cluster.

```
Client
   |
   v
Broker1 (bootstrap)
   |
   v
Cluster Metadata
(Broker1, Broker2, Broker3)
```

So even though you specified only:

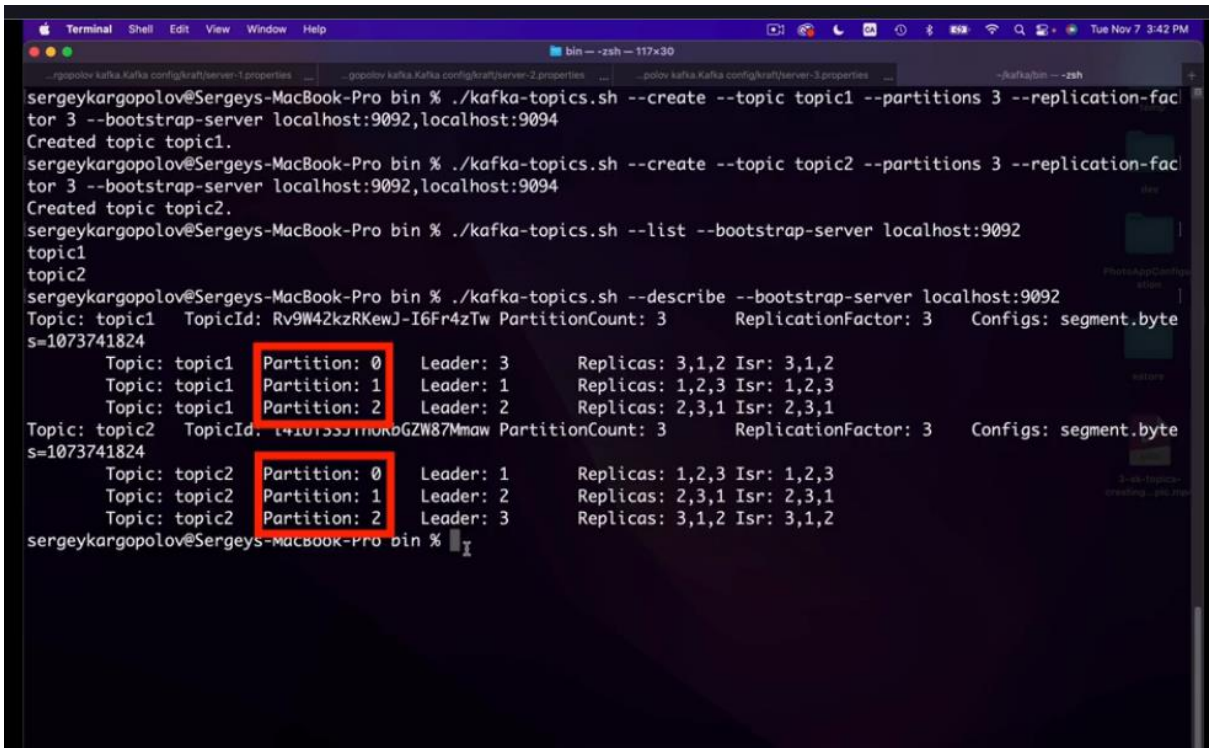
```
localhost:9092
```

Kafka still knows about Broker2 and Broker3.

Bootstrap servers are the initial Kafka broker addresses used by producers and consumers to connect to a Kafka cluster and fetch cluster metadata such

as brokers, topics, and partition leaders. They are only the starting point for discovery and do not need to be the broker that ultimately handles the messages.

LIST & DESCRIBE TOPIC:



LOG & SEGMENTS

Think of it like this:

```
Topic
  └── Partition
        └── Log
              ├── Segment-1
              ├── Segment-2
              └── Segment-3
```

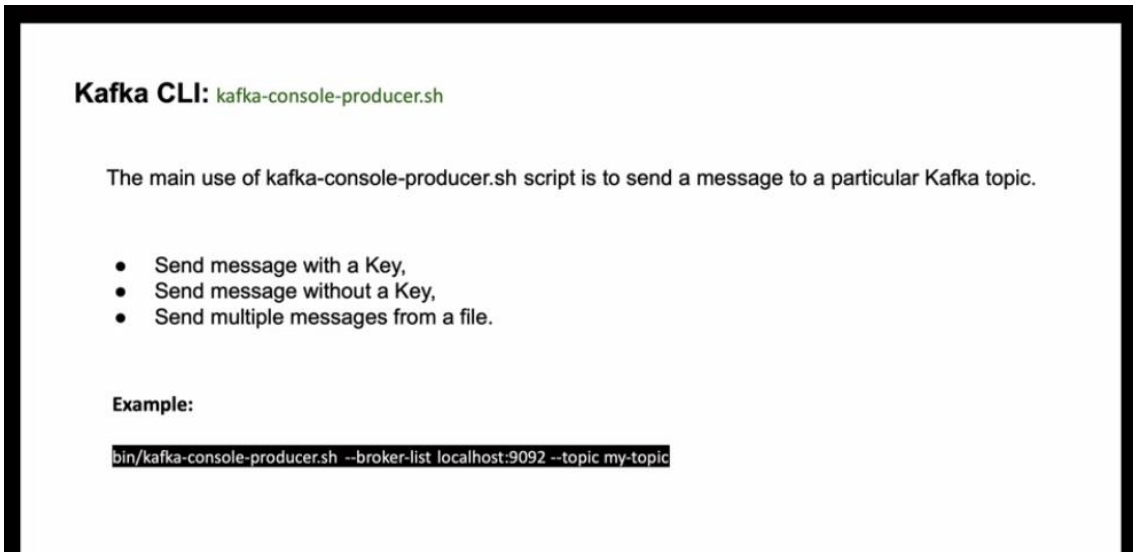
What is a Log?

A **log** is an ordered sequence of messages stored in a partition.

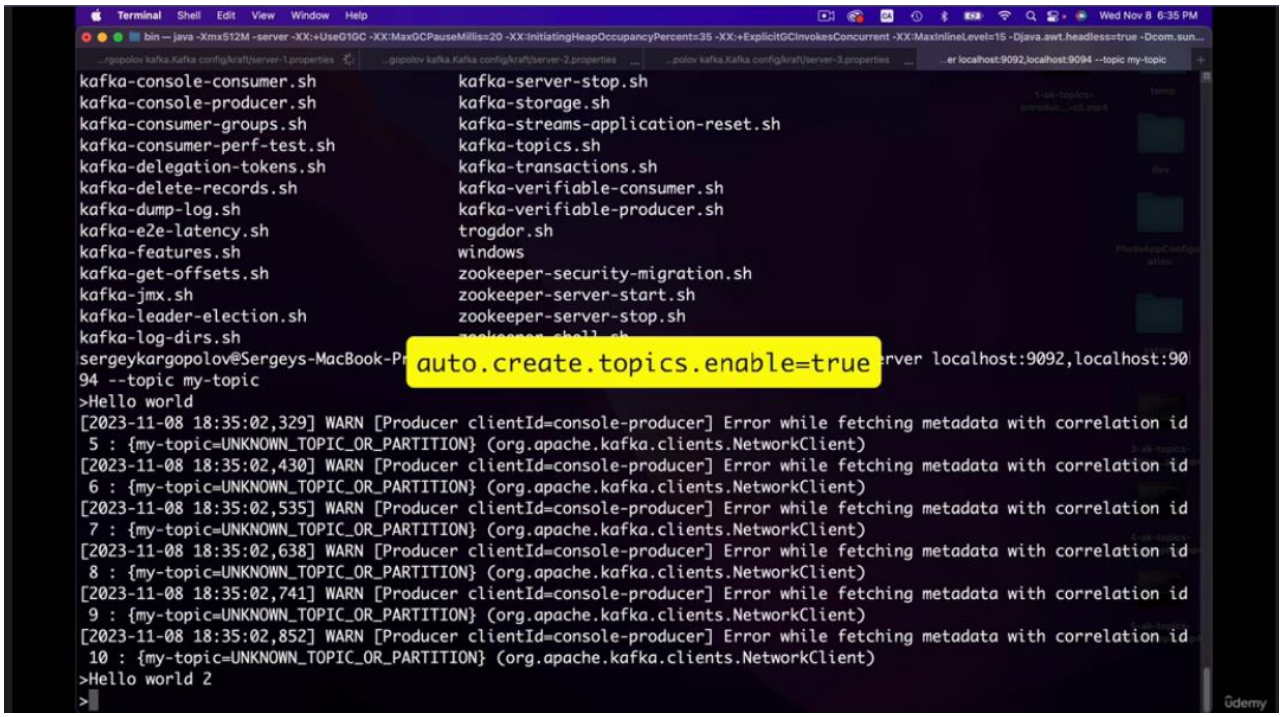
DELETE TOPIPC

```
sergeykargopolov@Sergeys-MacBook-Pro bin % ./kafka-topics.sh --delete --topic topic1 --bootstrap-server localhost:909
2
sergeykargopolov@Sergeys-MacBook-Pro bin % ./kafka-topics.sh --describe --bootstrap-server localhost:9092
Topic: topic2   TopicId: l41UT3SJThORbGZW87Mmaw PartitionCount: 3       ReplicationFactor: 3    Configs: segment.byte
s=1073741824
        Topic: topic2   Partition: 0    Leader: 1       Replicas: 1,2,3 Isr: 1,2,3
        Topic: topic2   Partition: 1    Leader: 2       Replicas: 2,3,1 Isr: 2,3,1
        Topic: topic2   Partition: 2    Leader: 3       Replicas: 3,1,2 Isr: 3,1,2
sergeykargopolov@Sergeys-MacBook-Pro bin %
```

KAFKA PRODUCERS:



By default this is true, If you try to send the message to a topic where the topic is not created then kafka automatically creates topic

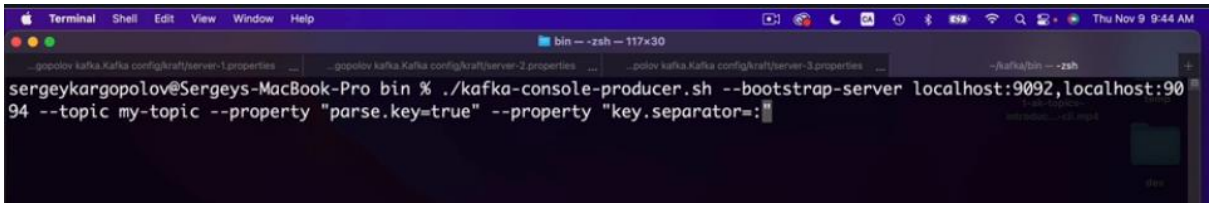


./kafka-console-producer.sh –bootstrap-server localhost:9092 –topic topic1

```
sergeykargopolov@Sergeys-MacBook-Pro bin % ./kafka-console-producer.sh --bootstrap-server localhost:9092,localhost:90
94 --topic my-topic
>Hello world
```

SEND MSG ALONG WITH KEY

/kafka-console-producer.sh –bootstrap-server localhost:9092 –-topic topic1 –property “parse.key=true” –property “key.separator=:”

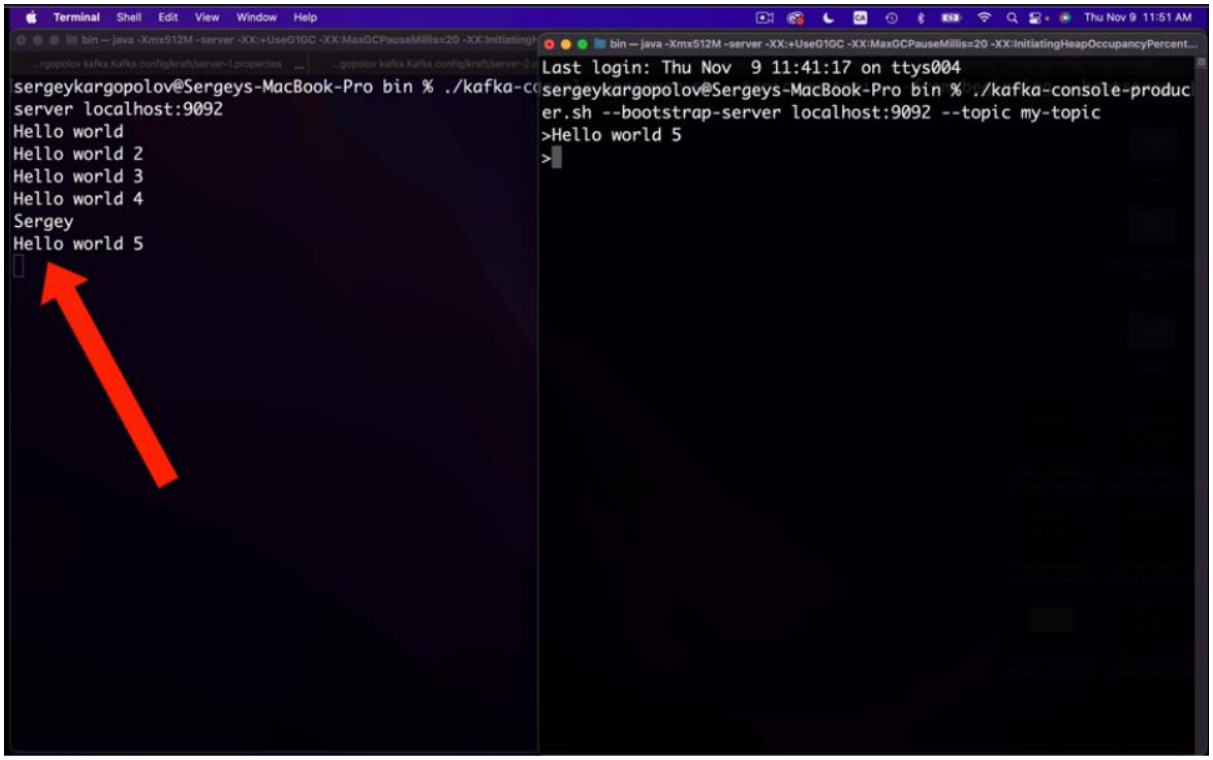


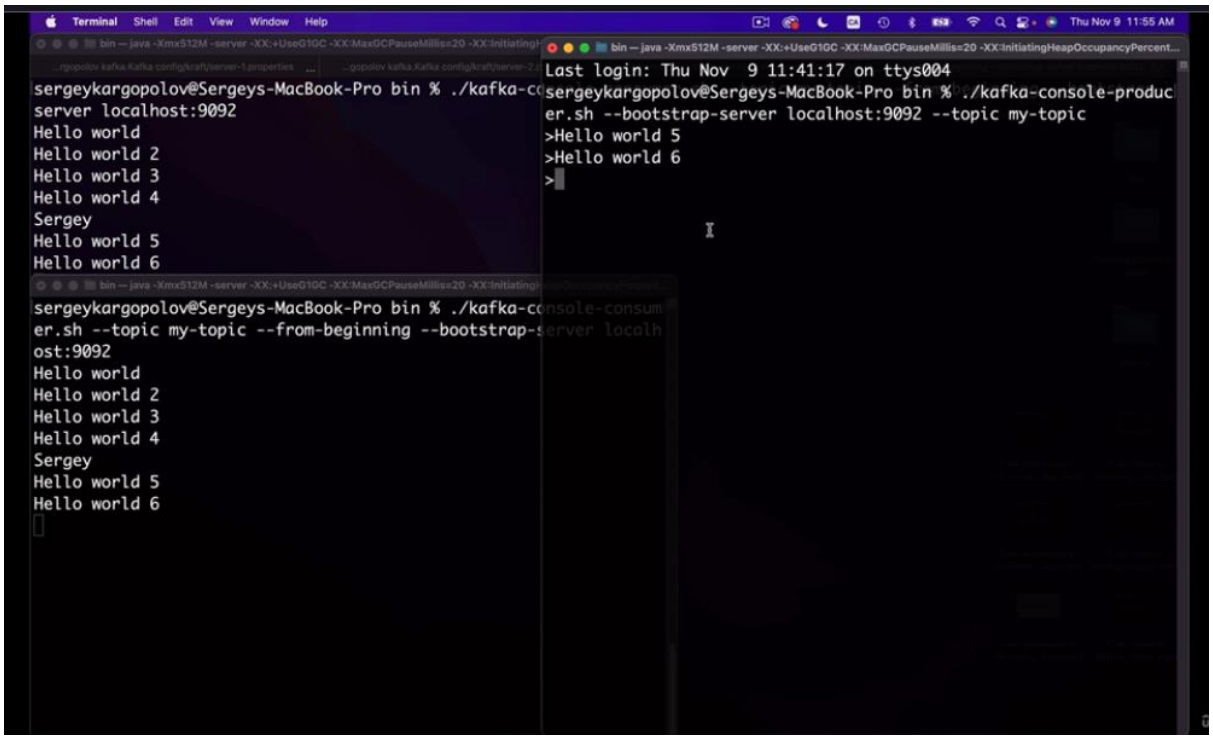
READ MSG IN CONSUMER



./kafka-console-consumer.sh –topic topic1 –from-beginning –bootstrap-server localhost:9092

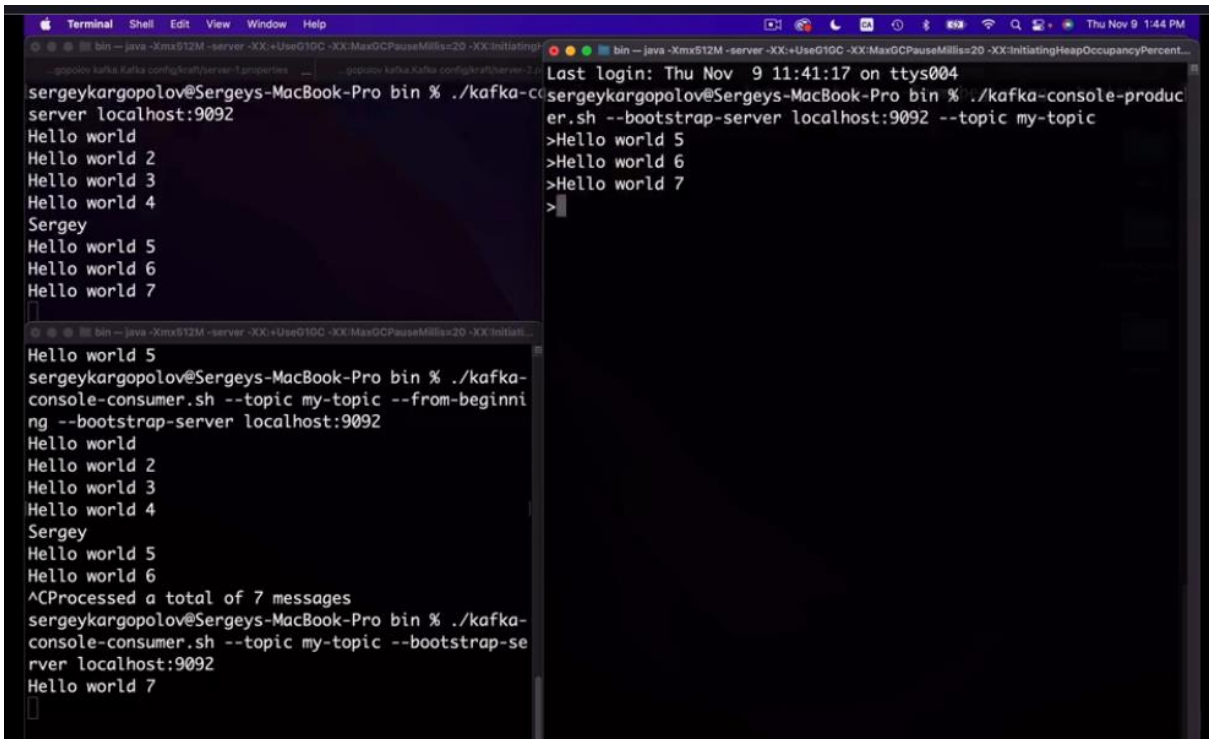



Terminal Shell Edit View Window Help
Thu Nov 9 11:51 AM
sergeykargopolov@Sergeys-MacBook-Pro bin % ./kafka-c
server localhost:9092
Hello world
Hello world 2
Hello world 3
Hello world 4
Sergey
Hello world 5
Last login: Thu Nov 9 11:41:17 on ttys004
sergeykargopolov@Sergeys-MacBook-Pro bin % ./kafka-console-produc
er.sh --bootstrap-server localhost:9092 --topic my-topic
>Hello world 5
>

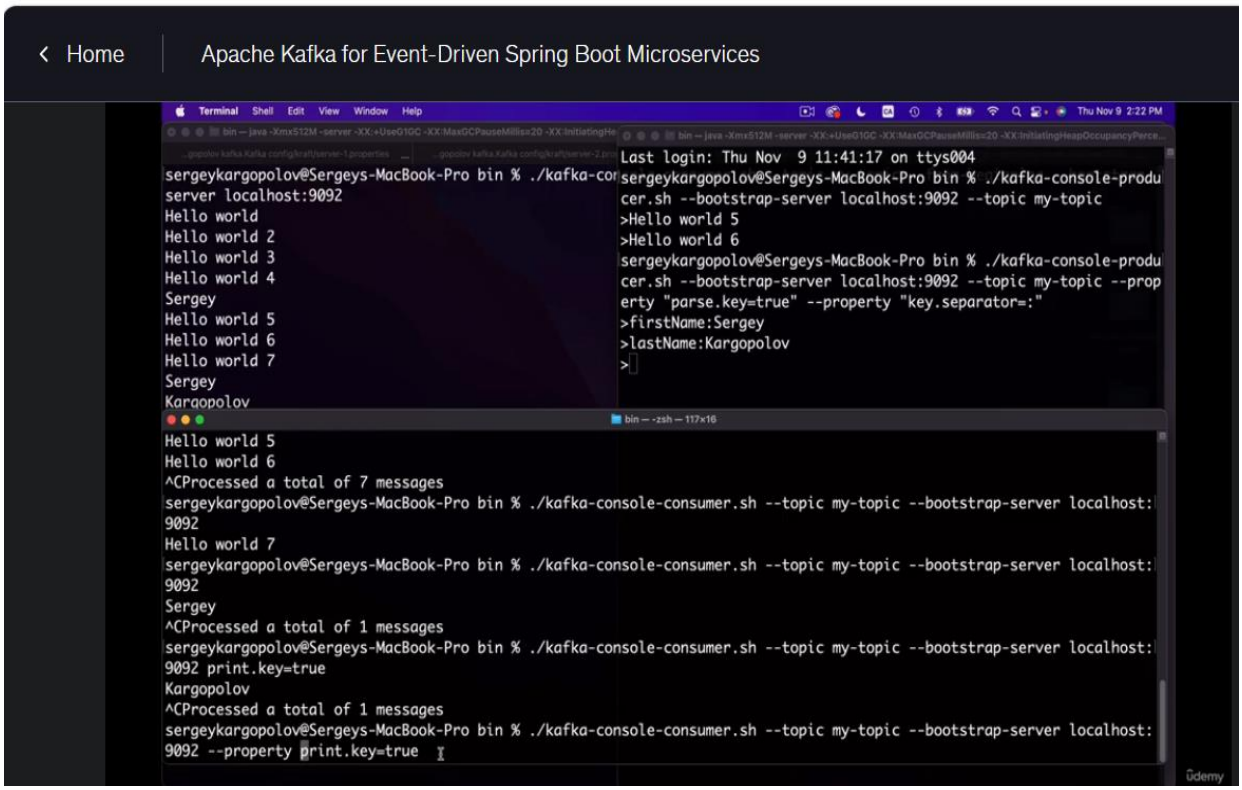

Terminal Shell Edit View Window Help
Thu Nov 9 11:55 AM
sergeykargopolov@Sergeys-MacBook-Pro bin % ./kafka-c
server localhost:9092
Hello world
Hello world 2
Hello world 3
Hello world 4
Sergey
Hello world 5
Hello world 6
Last login: Thu Nov 9 11:41:17 on ttys004
sergeykargopolov@Sergeys-MacBook-Pro bin % ./kafka-console-produc
er.sh --bootstrap-server localhost:9092 --topic my-topic
>Hello world 5
>Hello world 6
>
sergeykargopolov@Sergeys-MacBook-Pro bin % ./kafka-c
er.sh --topic my-topic --from-beginning --bootstrap-
ost:9092
Hello world
Hello world 2
Hello world 3
Hello world 4
Sergey
Hello world 5
Hello world 6

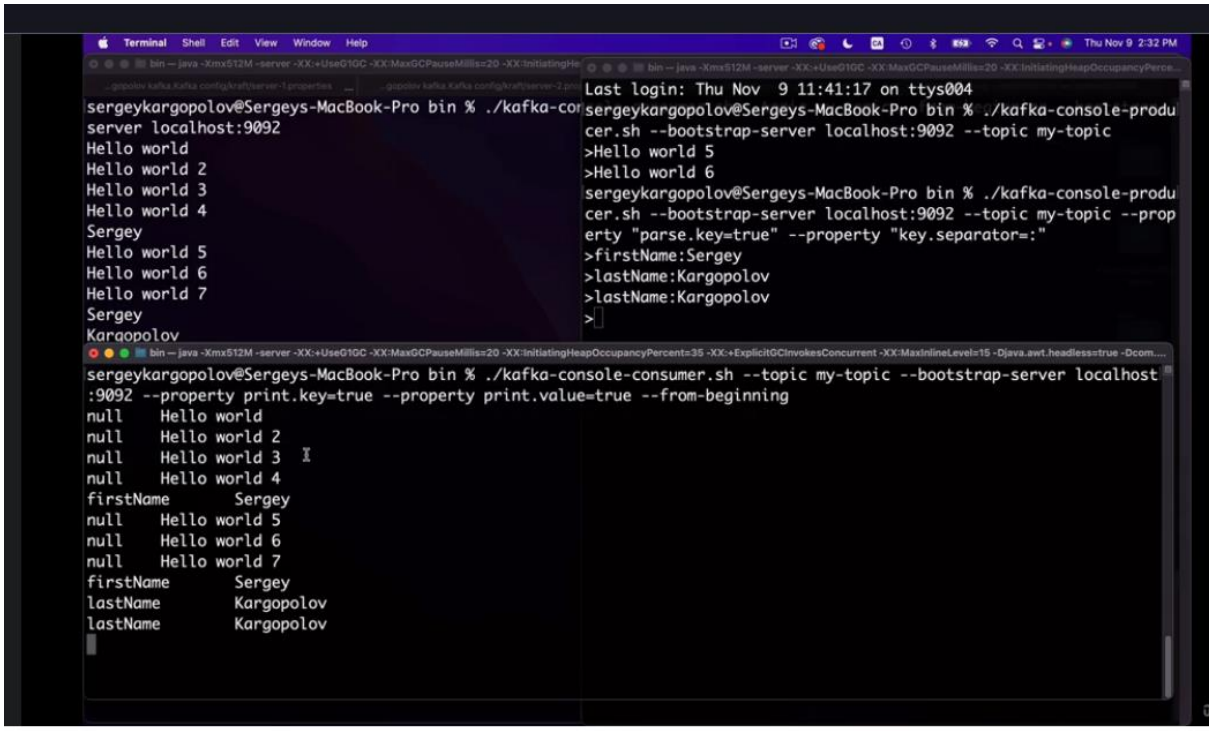
CONSUME NEW MSG ONLY

./kafka-console-consumer.sh –topic topic1 –bootstrap-server localhost:9092



Consumer by default just read the value if you want the consumer to read both value and key then use --property print.key=true





Why does `print.value=true` exist?

Because sometimes you may want to print **only the key**.

Example:

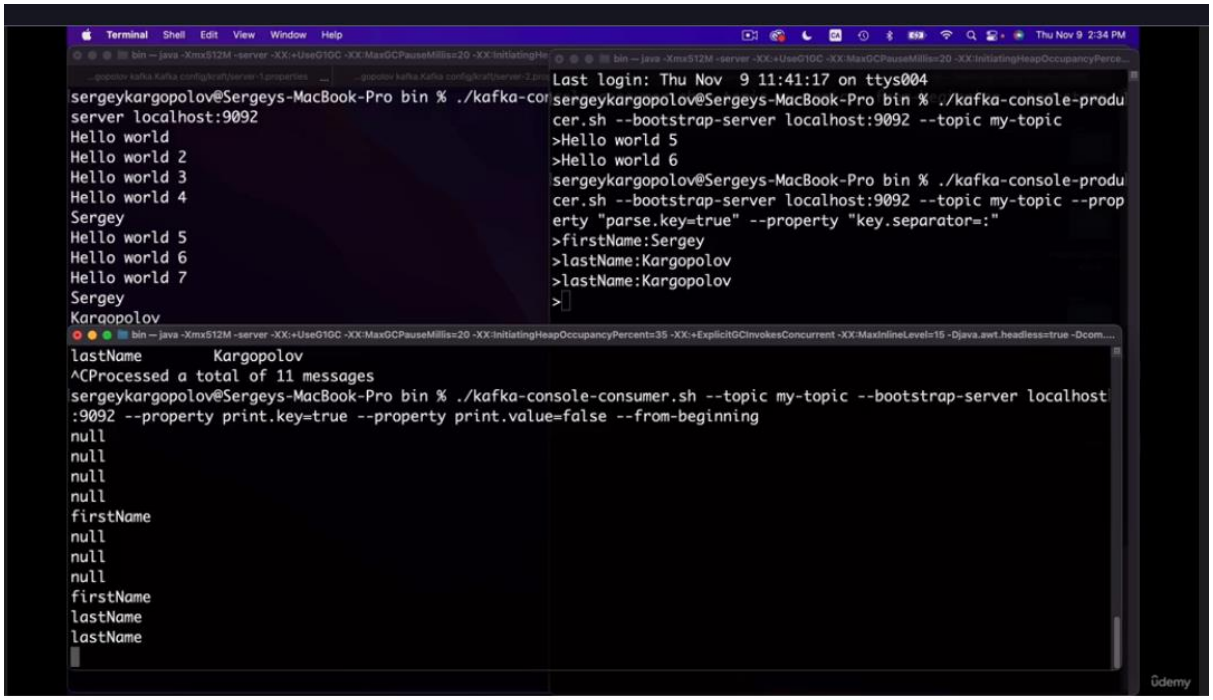
Shell

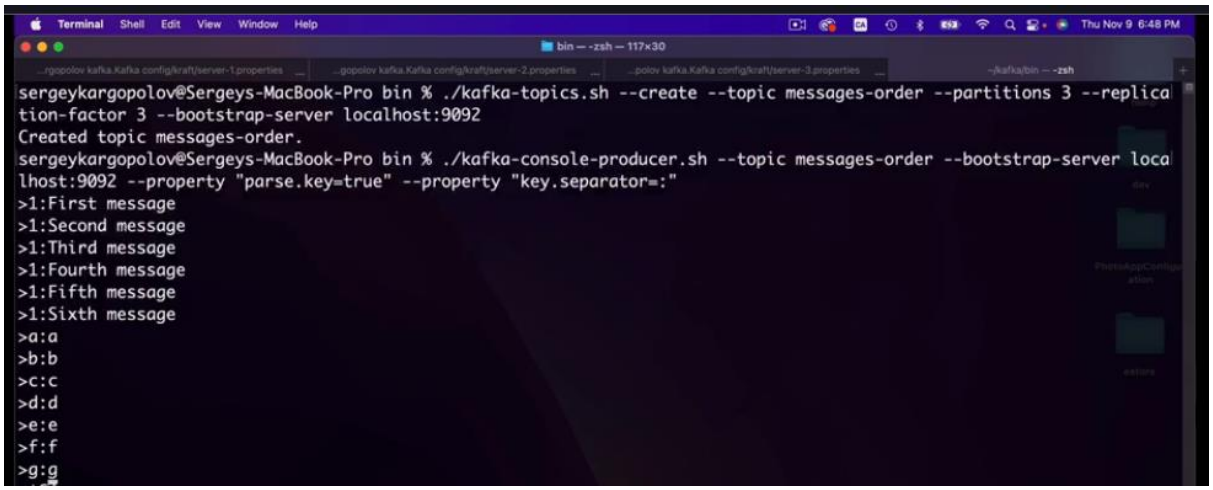
```
kafka-console-consumer.sh \
  --bootstrap-server localhost:9092 \
  --topic users \
  --property print.key=true \
  --property print.value=false
```

Output:

Plain Text

```
user123
```


Terminal Shell Edit View Window Help
Thu Nov 9 2:34 PM
sergeykargopolov@Sergeys-MacBook-Pro bin % ./kafka-cor
server localhost:9092
Hello world
Hello world 2
Hello world 3
Hello world 4
Sergey
Hello world 5
Hello world 6
Hello world 7
Sergey
Kargopolov
Last login: Thu Nov 9 11:41:17 on ttys004
sergeykargopolov@Sergeys-MacBook-Pro bin % ./kafka-console-produ
cer.sh --bootstrap-server localhost:9092 --topic my-topic
>Hello world 5
>Hello world 6
sergeykargopolov@Sergeys-MacBook-Pro bin % ./kafka-console-produ
cer.sh --bootstrap-server localhost:9092 --topic my-topic --prop
erty "parse.key=true" --property "key.separator=:"
>firstName:Sergey
>lastName:Kargopolov
>lastName:Kargopolov
lastName Kargopolov
^CProcessed a total of 11 messages
sergeykargopolov@Sergeys-MacBook-Pro bin % ./kafka-console-consumer.sh --topic my-topic --bootstrap-server localhost
:9092 --property print.key=true --property print.value=false --from-beginning
null
null
null
null
firstName
null
null
null
firstName
lastName
lastName
Udemy


Terminal Shell Edit View Window Help
Thu Nov 9 6:48 PM
bin — -zsh — 117×30
sergeykargopolov@Sergeys-MacBook-Pro bin % ./kafka-topics.sh --create --topic messages-order --partitions 3 --replica
tion-factor 3 --bootstrap-server localhost:9092
Created topic messages-order.
sergeykargopolov@Sergeys-MacBook-Pro bin % ./kafka-console-producer.sh --topic messages-order --bootstrap-server loca
lhost:9092 --property "parse.key=true" --property "key.separator=:"
>1:First message
>1:Second message
>1:Third message
>1:Fourth message
>1:Fifth message
>1:Sixth message
>a:a
>b:b
>c:c
>d:d
>e:e
>f:f
>g:g


sergeykargopolov@Sergeys-MacBook-Pro bin % ./kafka-console-consumer.sh --topic messages-order --bootstrap-server loca
lhost:9092 --from-beginning --property print.key=true
1 First message
1 Second message
1 Third message
1 Fourth message
1 Fifth message
1 Sixth message
f f
g g
a a
c c
b b
d d
e e

KAFKA PRODUCER USING SPRING BOOT APPLICATION:

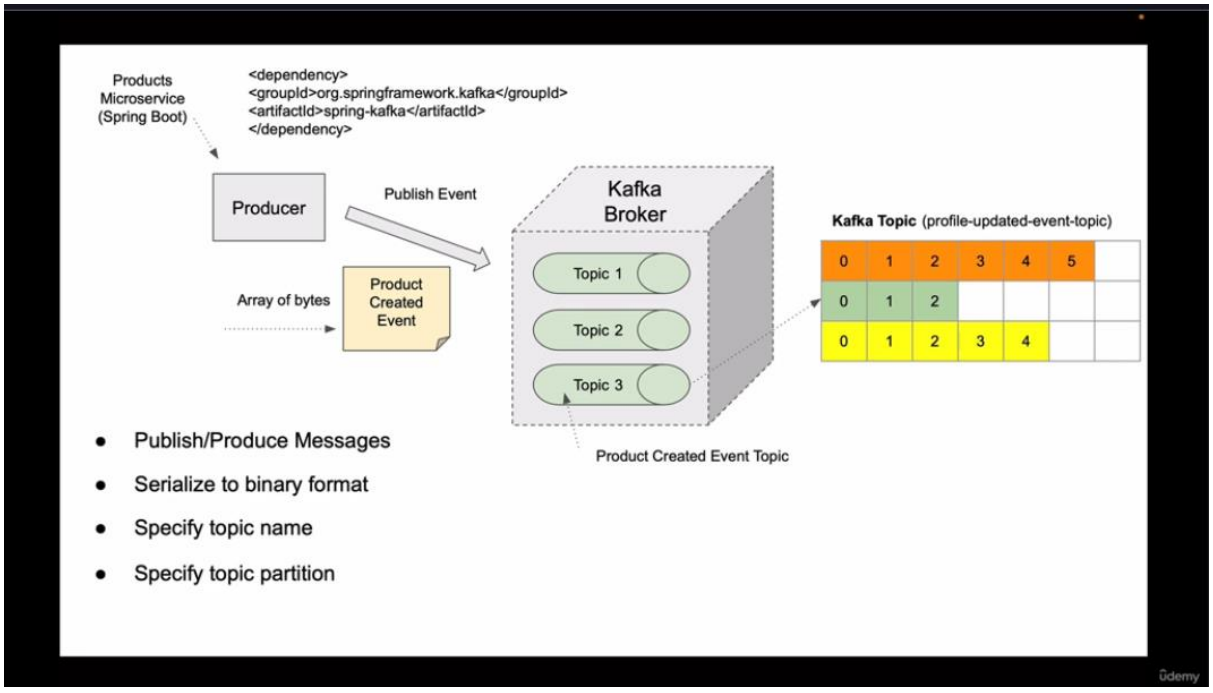
Responsibilities of a Kafka Producer (Spring Boot)

A Kafka Producer is responsible for:

1. **Publishing messages/events to Kafka topics**

```java
kafkaTemplate.send("orders-topic", order);
```

2. **Choosing the partition**
   - Kafka can automatically choose a partition.
   - Or you can provide a key and Kafka will determine the partition based on the key hash.
3. **Serialization**
   - Converts Java objects into bytes (JSON, Avro, String, etc.) before sending.
4. **Retry handling**
   - Retries sending messages when transient failures occur.
5. **Ensuring delivery guarantees**
   - At-most-once
   - At-least-once
   - Exactly-once (with proper configuration)



Can we send events to the same partition?

Yes, absolutely.

In fact, multiple events are commonly sent to the same partition.

Method 1: Use the same key

```java
kafkaTemplate.send("orders-topic", "customer-123", order1);
kafkaTemplate.send("orders-topic", "customer-123", order2);
kafkaTemplate.send("orders-topic", "customer-123", order3);
```

All messages with key `"customer-123"` will go to the **same partition** (assuming partition count doesn't change).

Example:

```
Partition 0
------------
customer-123 -> Order1
customer-123 -> Order2
customer-123 -> Order3
```

This preserves ordering.

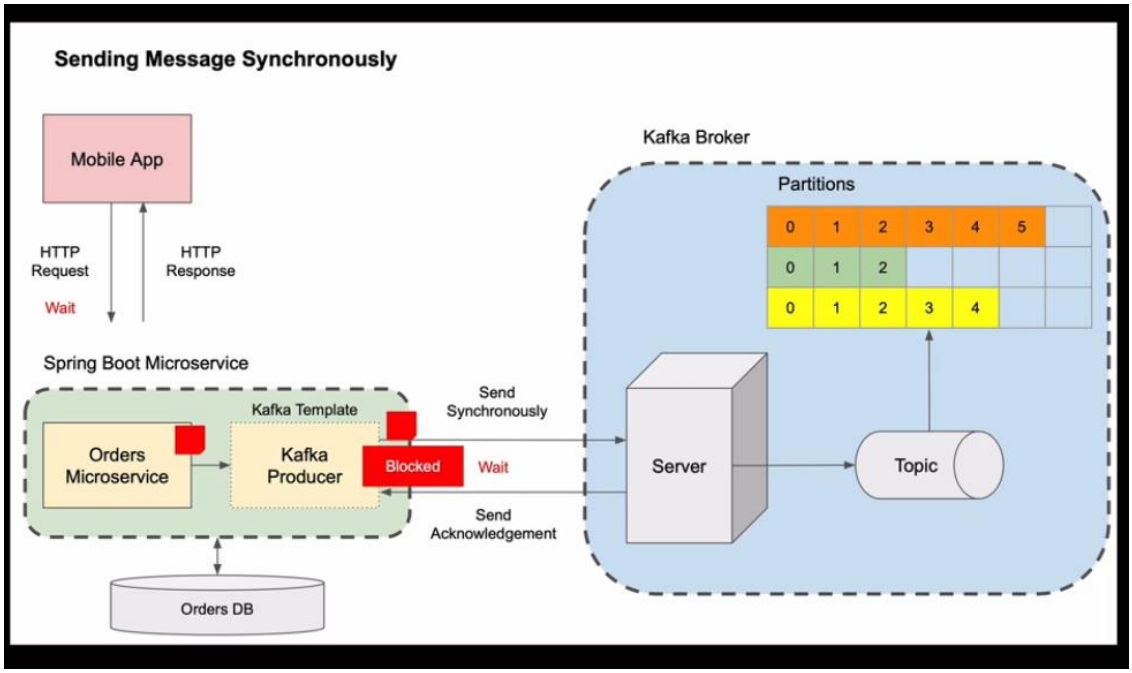
Method 2: Explicitly specify a partition

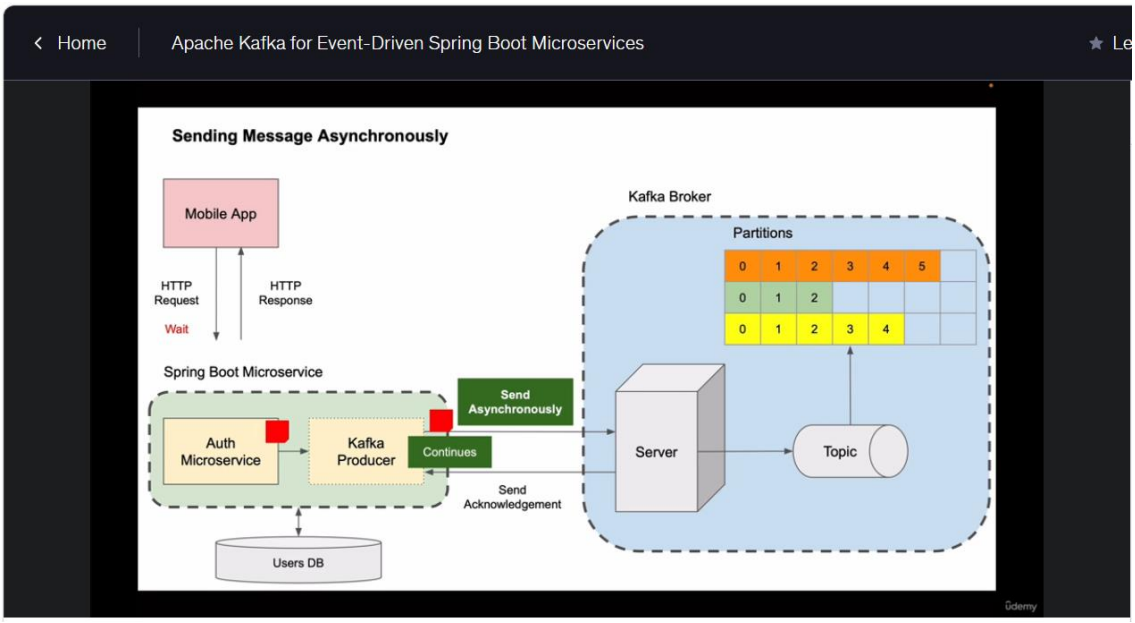
```java
kafkaTemplate.send(
    "orders-topic",
    2,
    "customer-123",
    order
);
```

Here Kafka sends the message directly to **Partition 2**.

Interview Answer

> Yes. A Kafka producer can send multiple events to the same partition. The common approach is to use the same message key, which causes Kafka to consistently route related events to the same partition, preserving their order. Alternatively, the producer can explicitly specify the partition number.





What you know

How does the Kafka Producer behave when sending a User LoggedIn Event to the Kafl

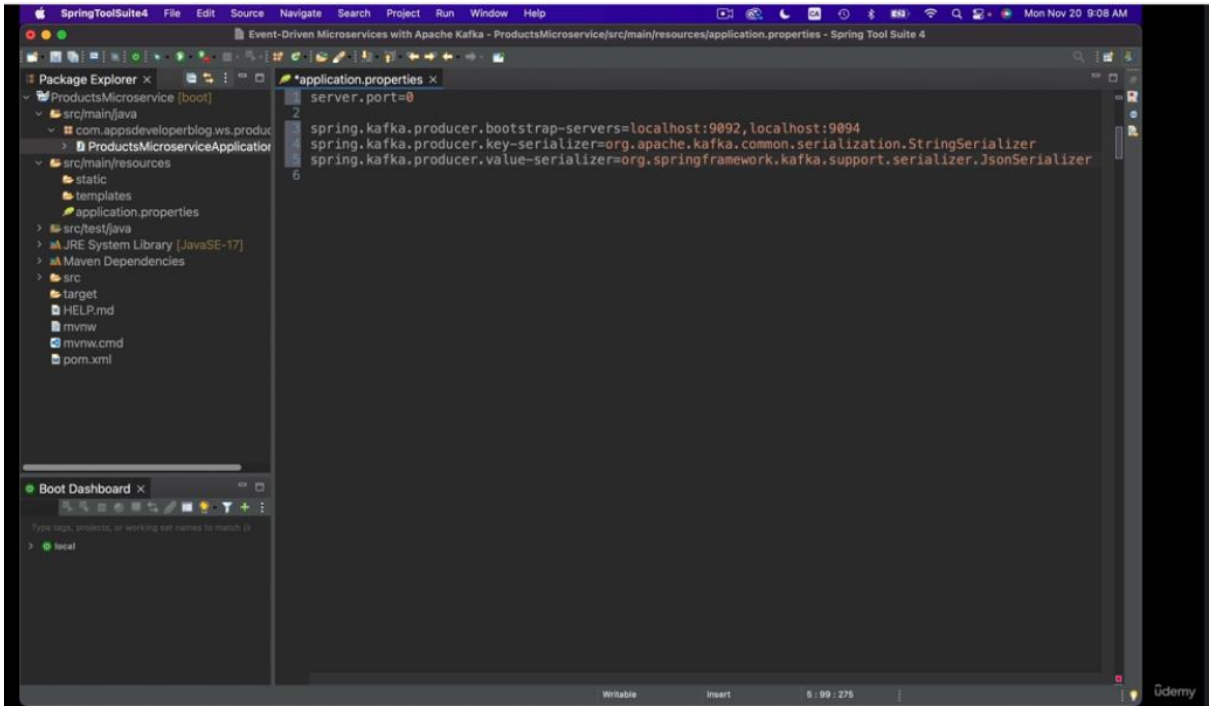
In the context of the Orders Microservice acting as a Kafka Producer, what characterize

In a Kafka Producer, what is the responsibility related to message serialization?

What library is typically used to integrate Kafka functionality into a Spring Boot applica

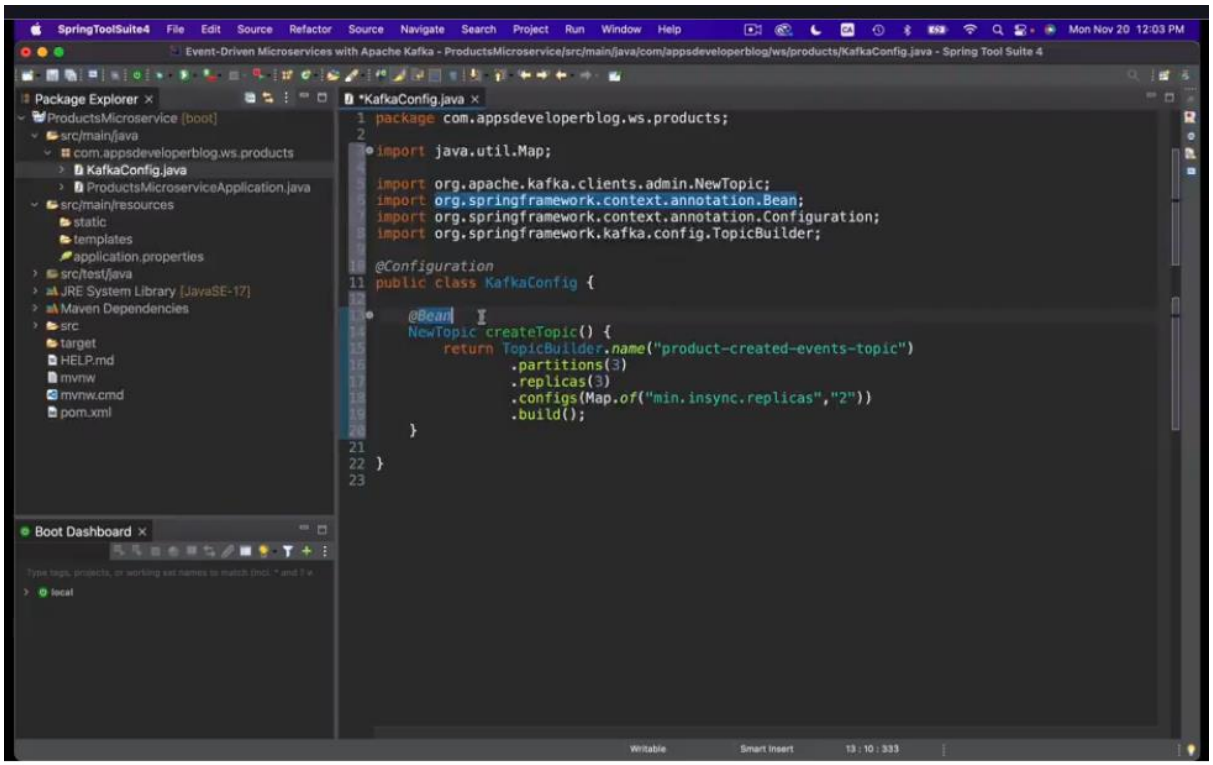
How does a Kafka Producer determine which partition to send a message to?

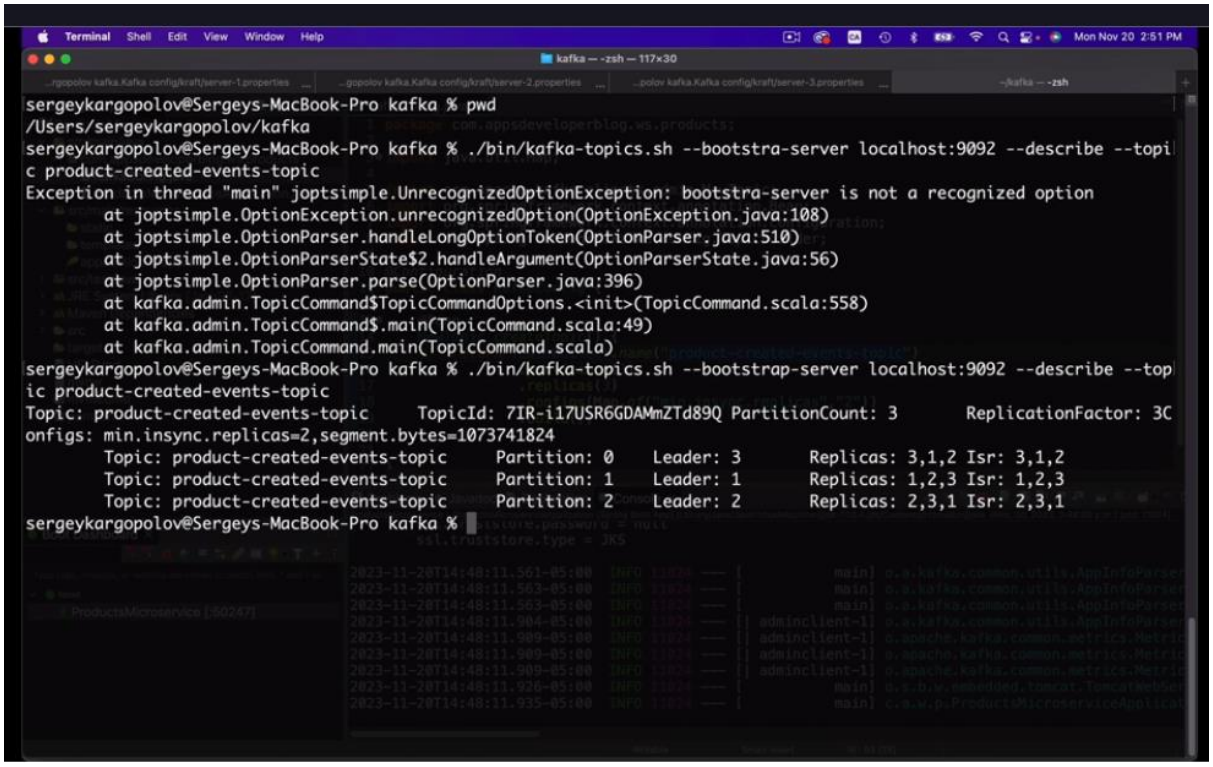
What is the primary function of a Kafka Producer?



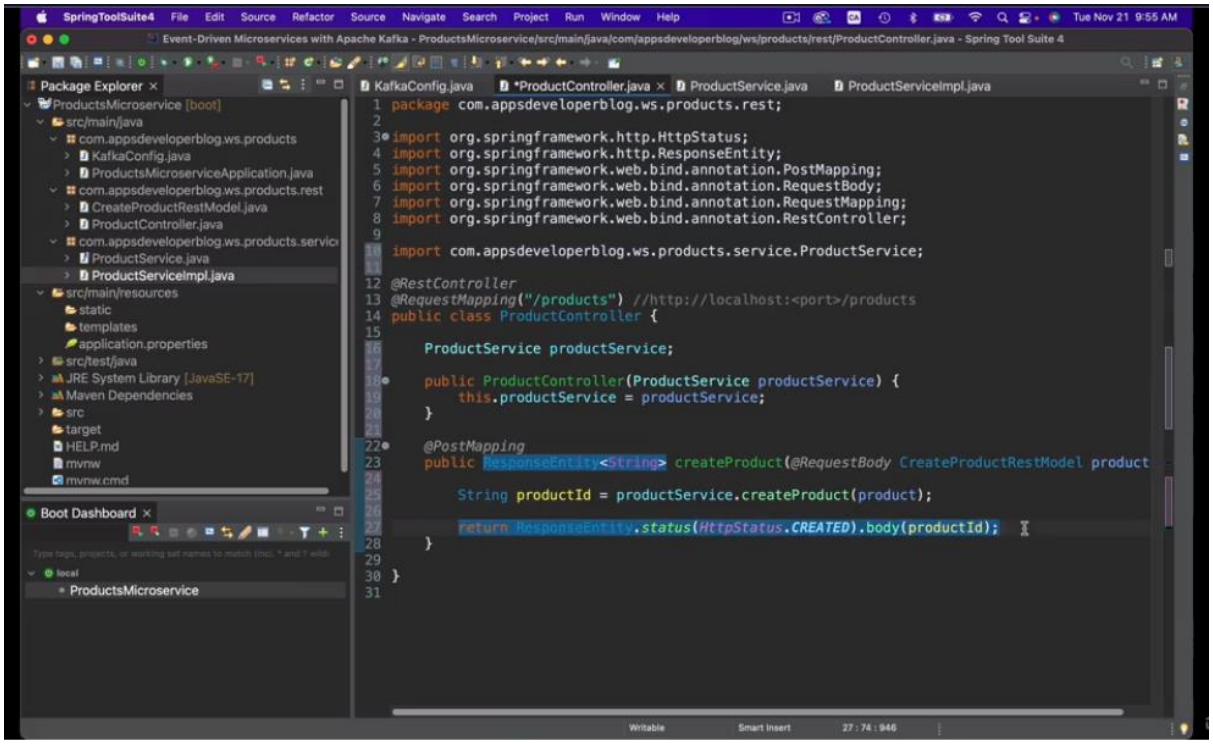
Partition = parallel processing and scalability (Increase Throughput)

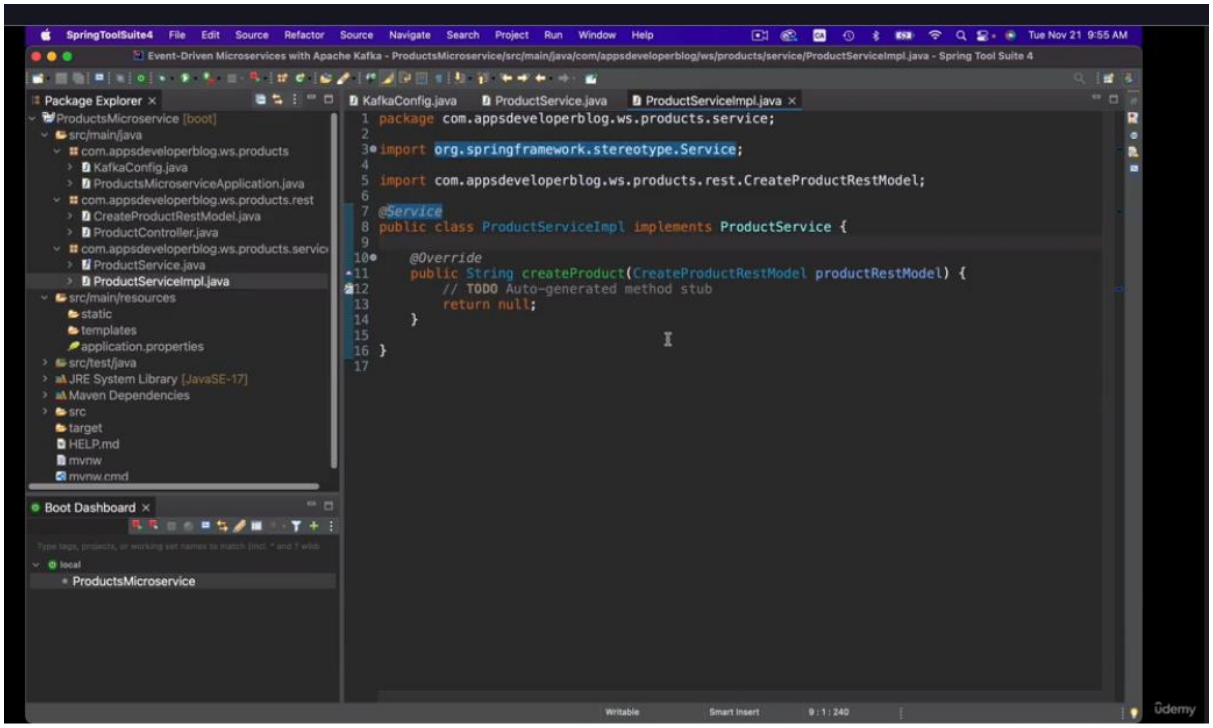
Replicas = fault tolerance

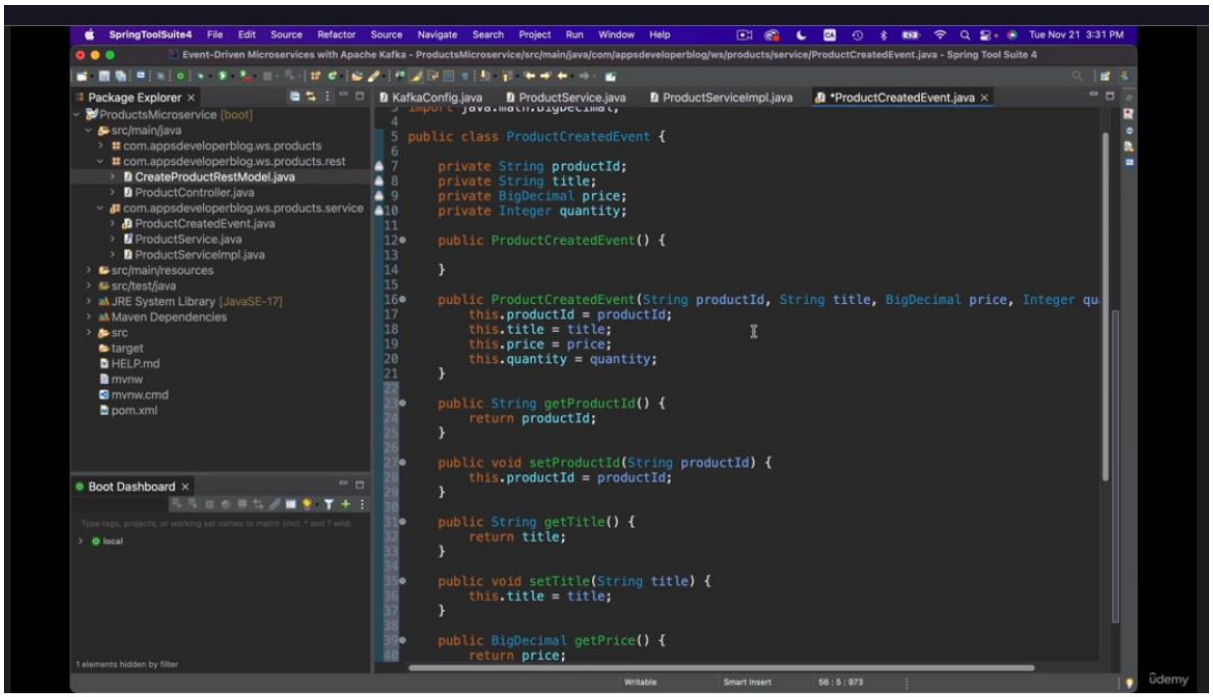




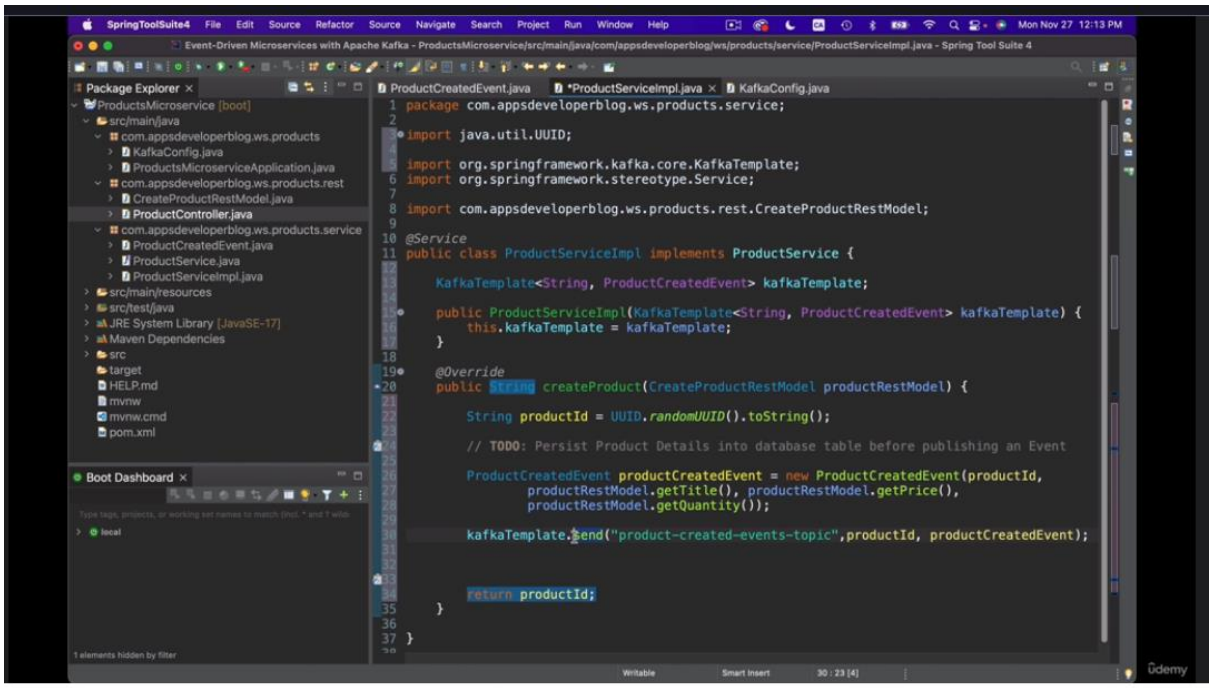
CREATING CONTROLLER / SERVICE LAYER

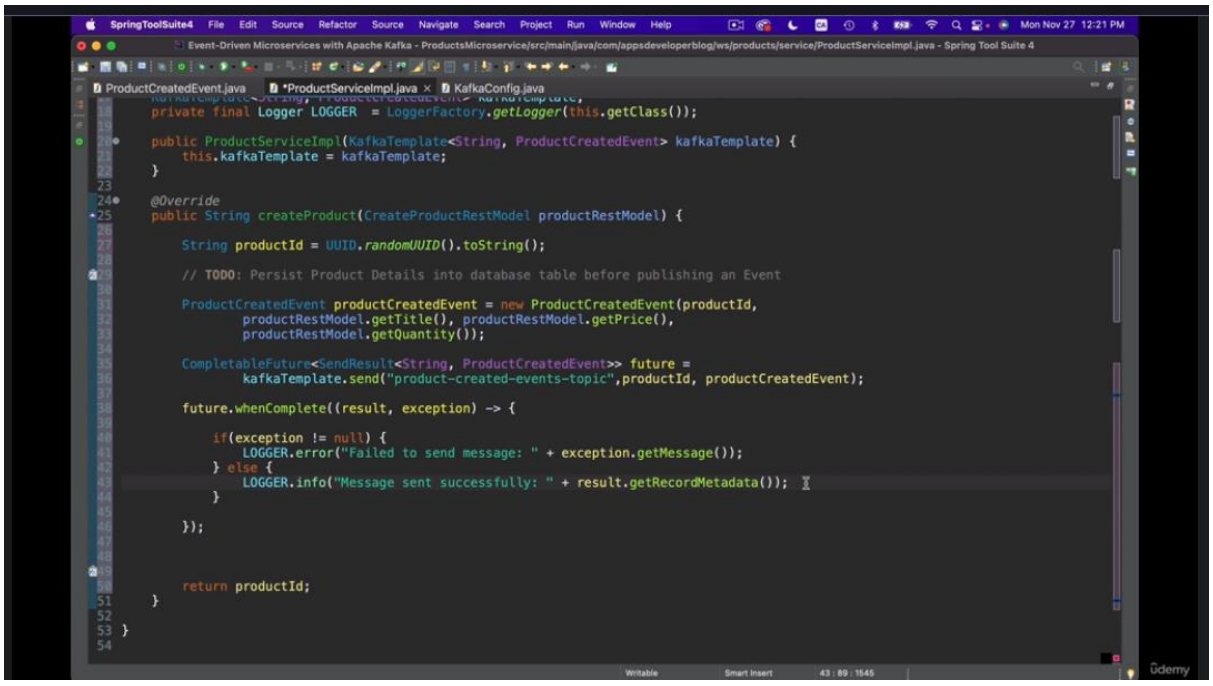


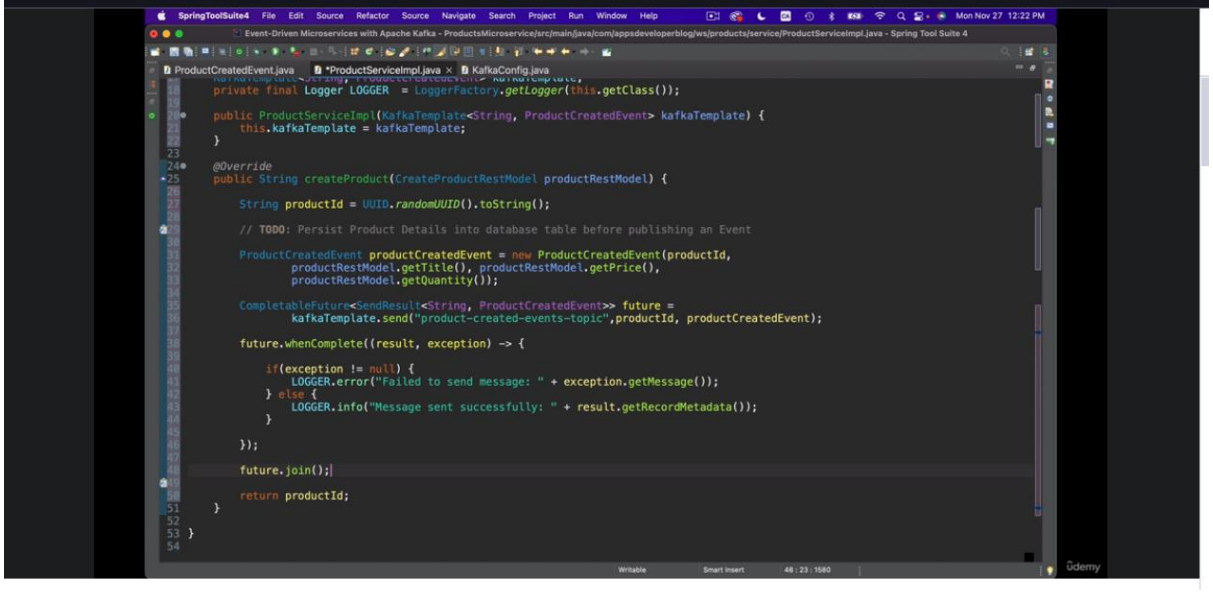

SpringToolSuite4 File Edit Source Refactor Source Navigate Search Project Run Window Help
Tue Nov 21 9:55 AM
Event-Driven Microservices with Apache Kafka - ProductsMicroservice/src/main/java/com/appsdeveloperblog/ws/products/service/ProductServiceImpl.java - Spring Tool Suite 4
Package Explorer
ProductsMicroservice [boot]
src/main/java
com.appsdeveloperblog.ws.products
KafkaConfig.java
ProductsMicroserviceApplication.java
com.appsdeveloperblog.ws.products.rest
CreateProductRestModel.java
ProductController.java
com.appsdeveloperblog.ws.products.service
ProductService.java
ProductServiceImpl.java
src/main/resources
static
templates
application.properties
src/test/java
JRE System Library [JavaSE-17]
Maven Dependencies
src
target
HELP.md
mvnw
mvnw.cmd
Boot Dashboard
local
ProductsMicroservice
KafkaConfig.java
ProductService.java
ProductServiceImpl.java
package com.appsdeveloperblog.ws.products.service;
import org.springframework.stereotype.Service;
import com.appsdeveloperblog.ws.products.rest.CreateProductRestModel;
@Service
public class ProductServiceImpl implements ProductService {
@Override
public String createProduct(CreateProductRestModel productRestModel) {
// TODO Auto-generated method stub
return null;
}
}
Writable
Smart Insert
9 : 1 : 240
Udemy


SpringToolSuite4 File Edit Source Refactor Source Navigate Search Project Run Window Help
Tue Nov 21 3:31 PM
Event-Driven Microservices with Apache Kafka - ProductsMicroservice/src/main/java/com/appsdeveloperblog/ws/products/service/ProductCreatedEvent.java - Spring Tool Suite 4
Package Explorer
ProductsMicroservice [boot]
src/main/java
com.appsdeveloperblog.ws.products
com.appsdeveloperblog.ws.products.rest
CreateProductRestModel.java
ProductController.java
com.appsdeveloperblog.ws.products.service
ProductCreatedEvent.java
ProductService.java
ProductServiceImpl.java
src/main/resources
src/test/java
JRE System Library [JavaSE-17]
Maven Dependencies
src
target
HELP.md
mvnw
mvnw.cmd
pom.xml
Boot Dashboard
local
1 elements hidden by filter
KafkaConfig.java
ProductService.java
ProductServiceImpl.java
*ProductCreatedEvent.java
public class ProductCreatedEvent {
private String productId;
private String title;
private BigDecimal price;
private Integer quantity;
public ProductCreatedEvent() {
}
public ProductCreatedEvent(String productId, String title, BigDecimal price, Integer qu
this.productId = productId;
this.title = title;
this.price = price;
this.quantity = quantity;
}
public String getProductId() {
return productId;
}
public void setProductId(String productId) {
this.productId = productId;
}
public String getTitle() {
return title;
}
public void setTitle(String title) {
this.title = title;
}
public BigDecimal getPrice() {
return price;
Writable
Smart Insert
56 : 5 : 973
Udemy

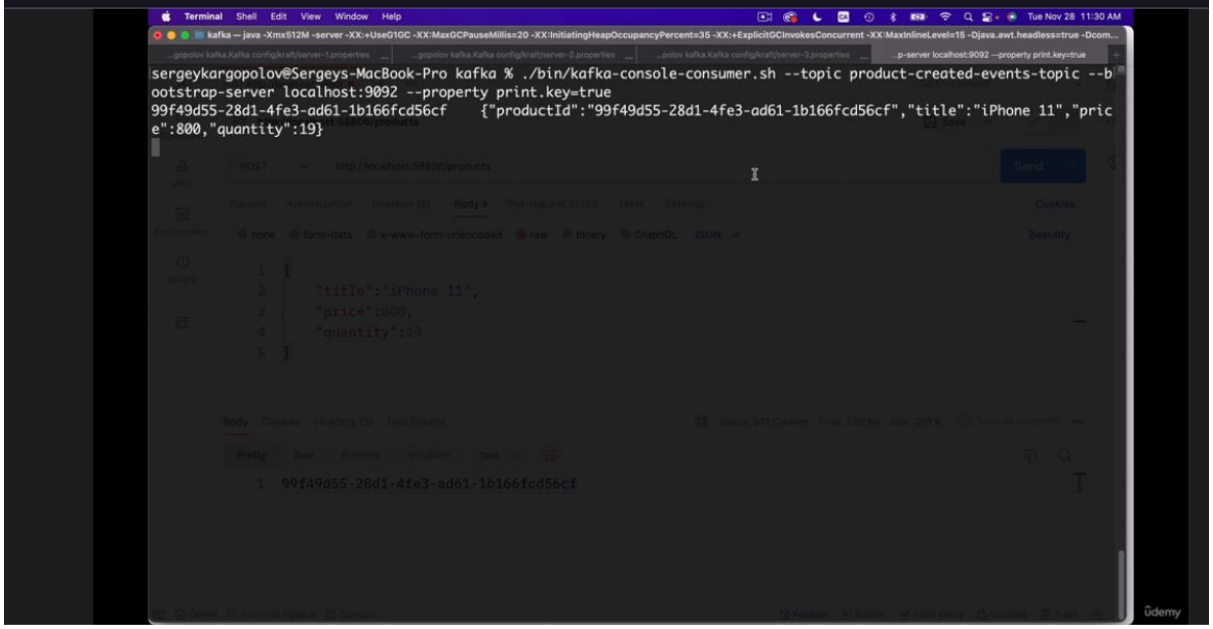
SEND MESSAGE SYNCHRONOUSLY



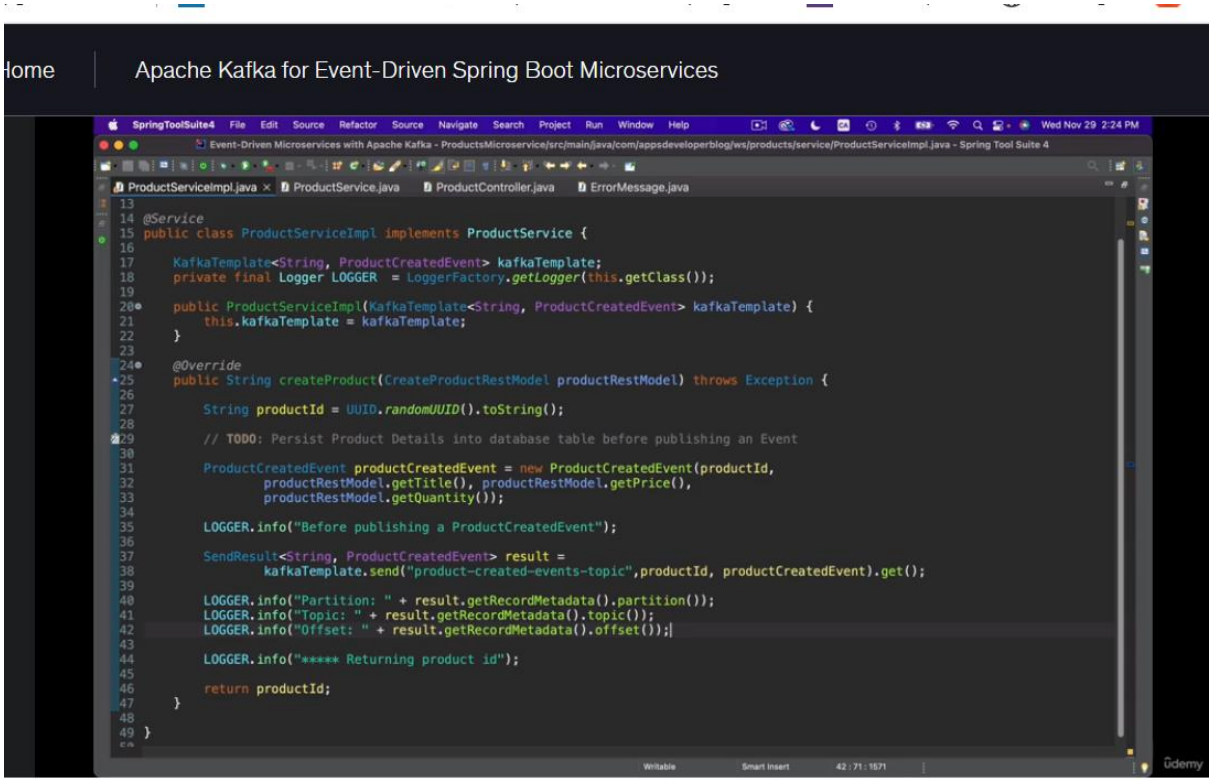


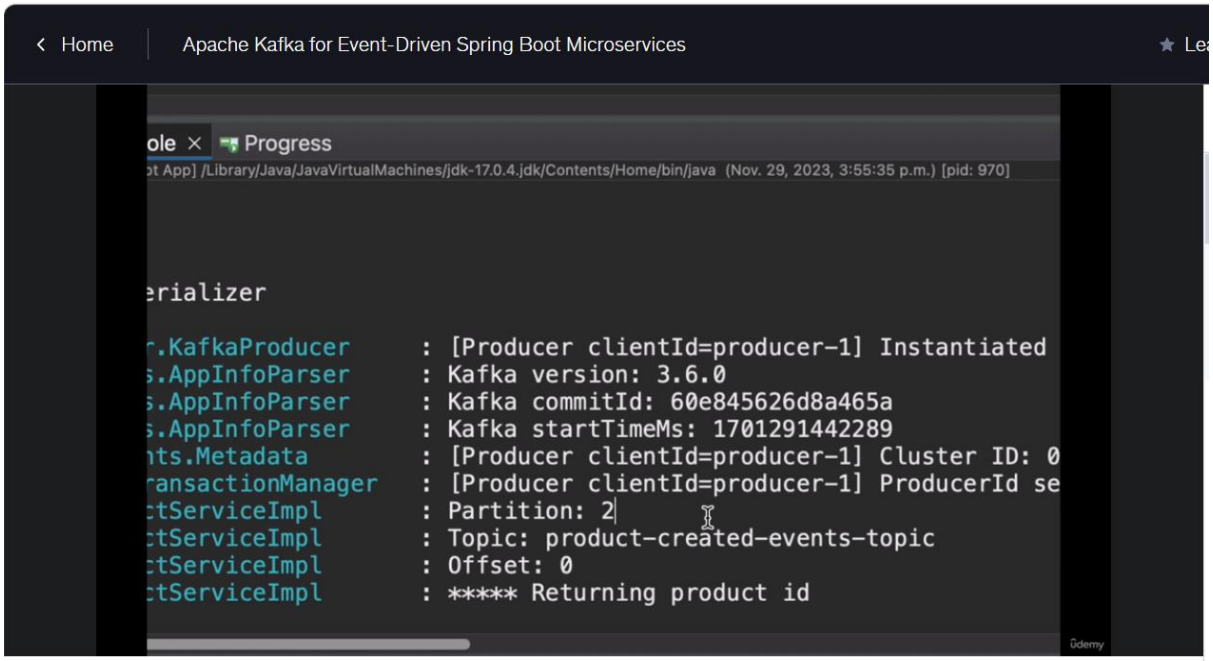


Home | Apache Kafka for Event-Driven Spring Boot Microservices

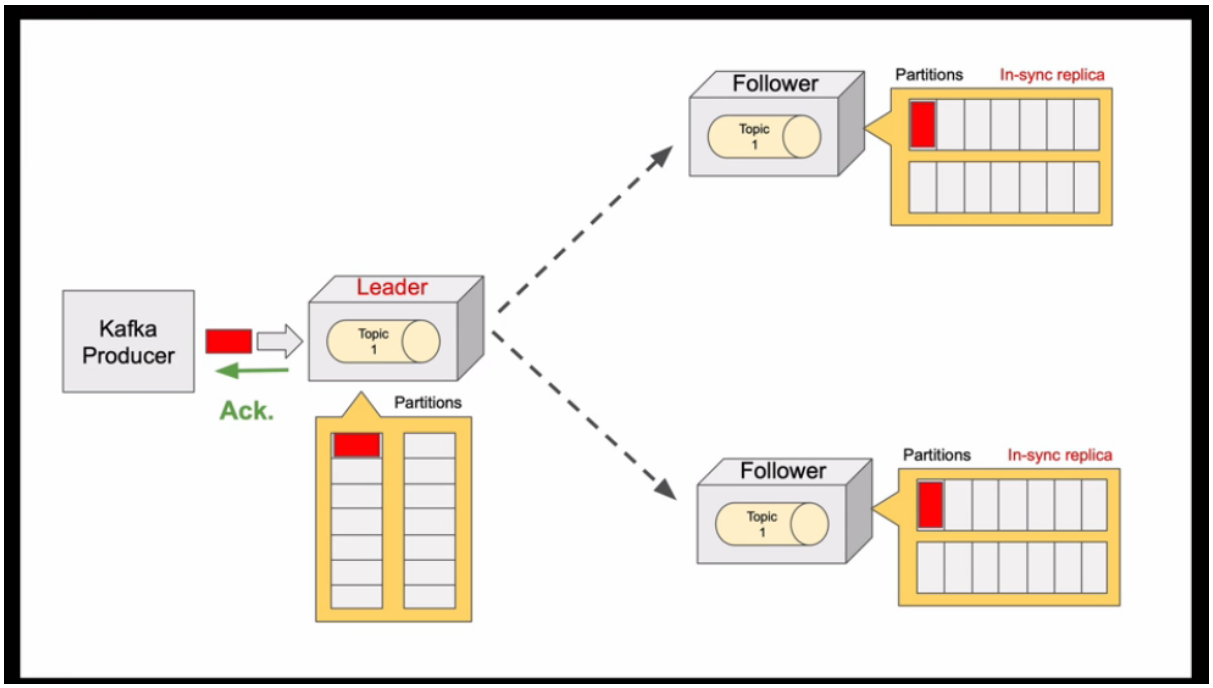


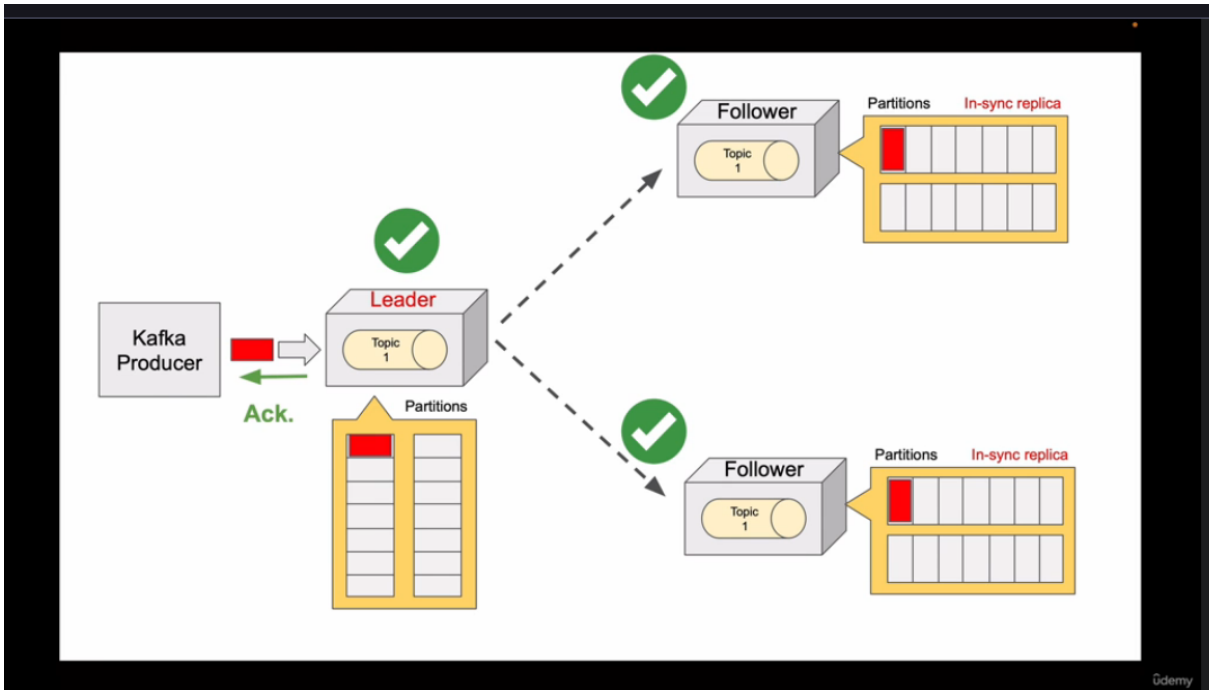
SEND MESSAGE SYNCHRONOUSLY:



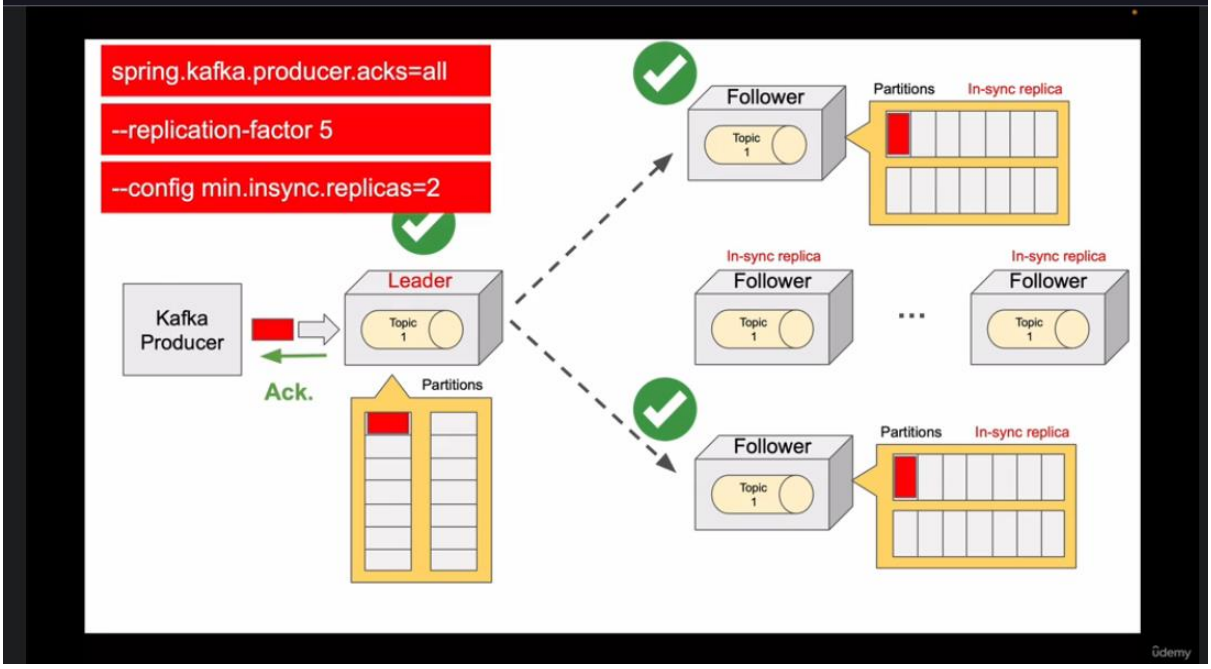


KAFKA PRODUCER ACKNOWLEDGEMENT





- **spring.kafka.producer.acks=all**
  Waits for an acknowledgement from all brokers.
- **spring.kafka.producer.acks=1**
  Waits for an acknowledgement from a leader broker only.
- **spring.kafka.producer.acks=0**
  Does not wait for an acknowledgement.

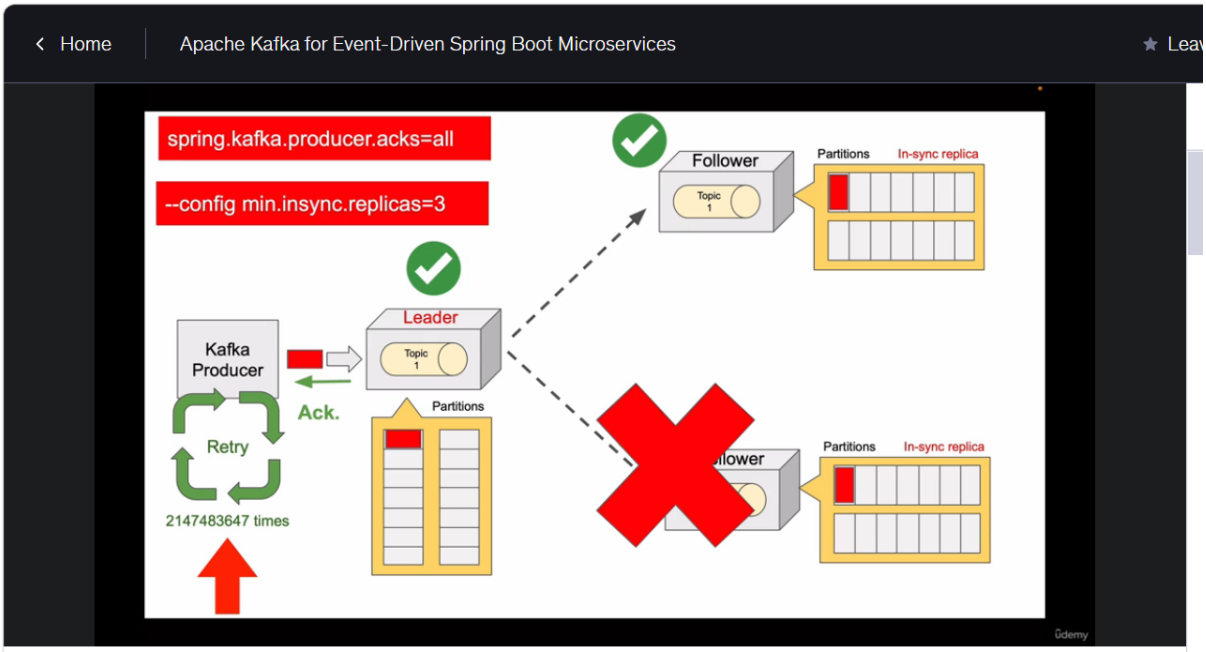


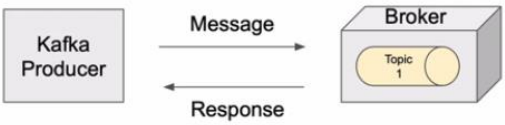
- **spring.kafka.producer.acks=all**
  Waits for an acknowledgement from all brokers.

--replication-factor 5

--config min.insync.replicas=2

Udemy





- **No response** - if the producer is configured with acks=0
- Acknowledgement (ACK) of Successful Storage
- **Non-Retryable Error** - A permanent problem that is unlikely to be resolved by retries.
- **Retryable Error** - A temporary problem that can be resolved by retrying the send operation.

Udemy

spring.kafka.producer.**retries**=10

How many times Kafka Producer will try to send a message before marking it as failed. Default value is 2147483647.

spring.kafka.producer.properties.**retry.backoff.ms**=1000

How long the producer will wait before attempting to retry a failed request. Default value is 100 ms.

Includes retry attempts

spring.kafka.producer.properties.**delivery.timeout.ms**=120000

The maximum time Producer can spend trying to deliver the message. Default value is 120000 ms (2 minutes).

delivery.timeout.ms >= linger.ms + request.timeout.ms

spring.kafka.producer.properties.**linger.ms**=0

The maximum time in milliseconds that the producer will wait and buffer data before sending a batch of messages. The default value is 0.

spring.kafka.producer.properties.**request.timeout.ms**=30000

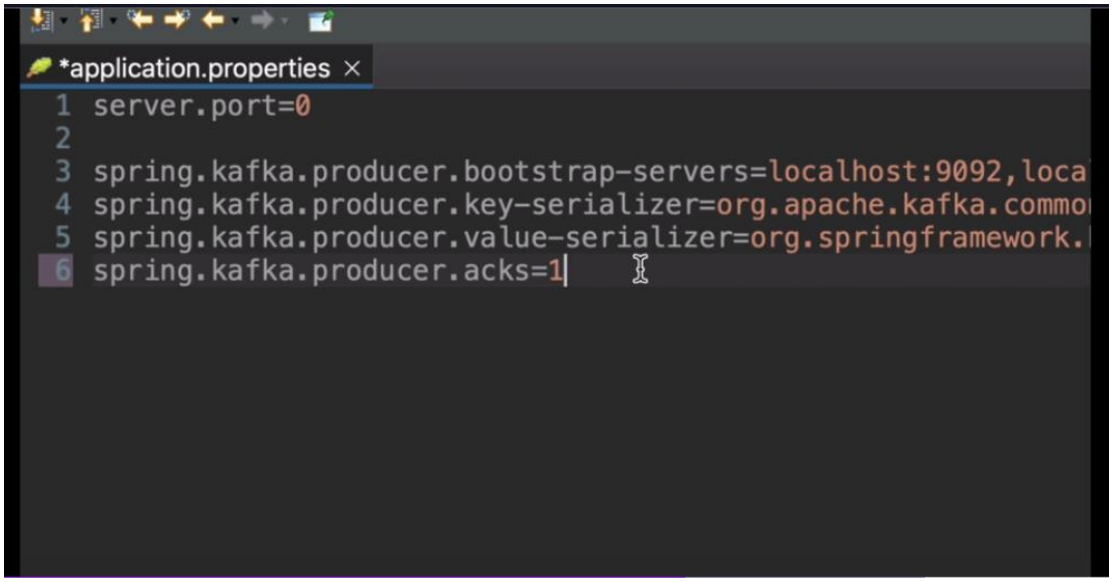
The maximum time to wait for a response fr[illegible]e broker after sending a request. The default value is 30000 ms.

Single request

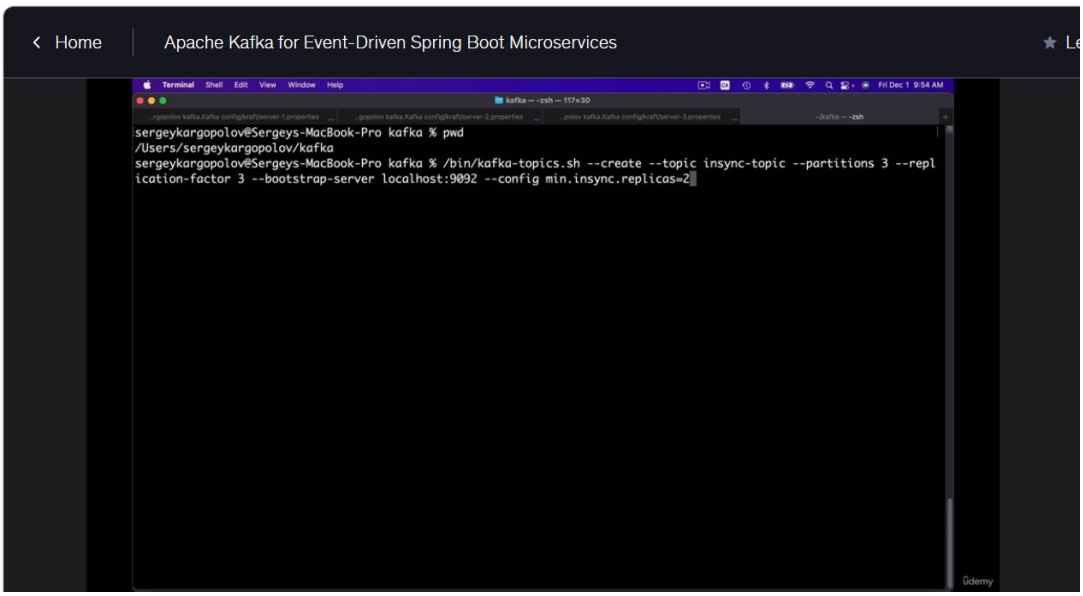
Udemy

Property	Meaning
linger.ms	Wait before sending batch
request.timeout.ms	Wait for one broker response
retries	Number of retry attempts
retry.backoff.ms	Gap between retries
delivery.timeout.ms	Total time allowed for the message

KAFKA PRODUCER ACK BY SPRING BOOT (By default 1)



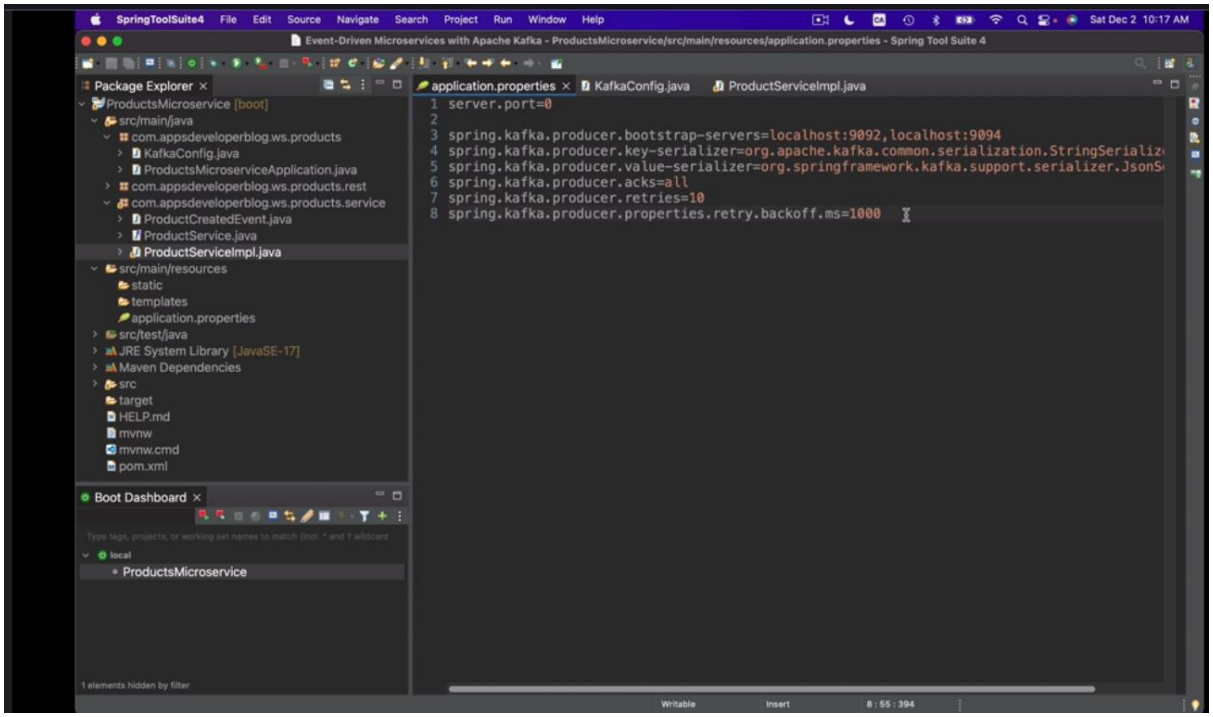
Set min-in-sync-replica in topic while creating the topic

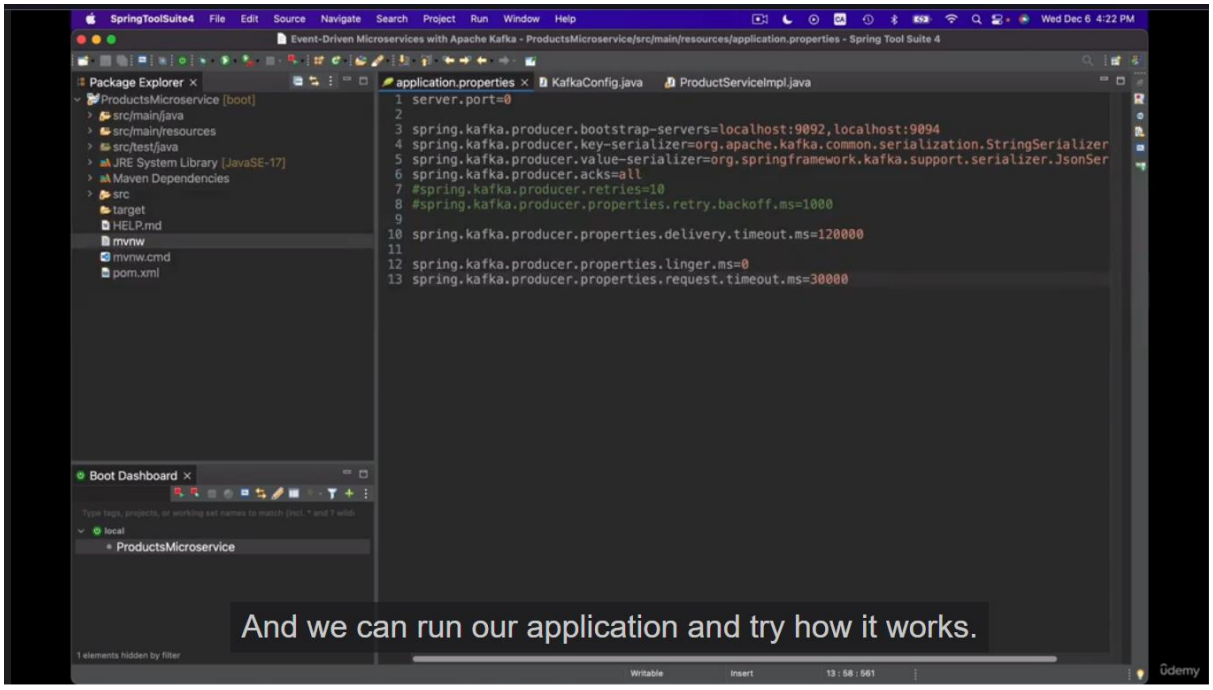


Set min-in-sync-replica in topic when the topic is already created

```
sergeykargopolov@Sergeys-MacBook-Pro kafka % ./bin/kafka-configs.sh --bootstrap-server localhost:9092 --alter --entit
y-type topics --entity-name topic2 --add-config min.insync.replicas=2
Completed updating config for topic topic2.
sergeykargopolov@Sergeys-MacBook-Pro kafka %
```

Retries and retry backoff ms





✓ What you know

What is a possible drawback of setting 'acks=0' in a Kafka Producer?

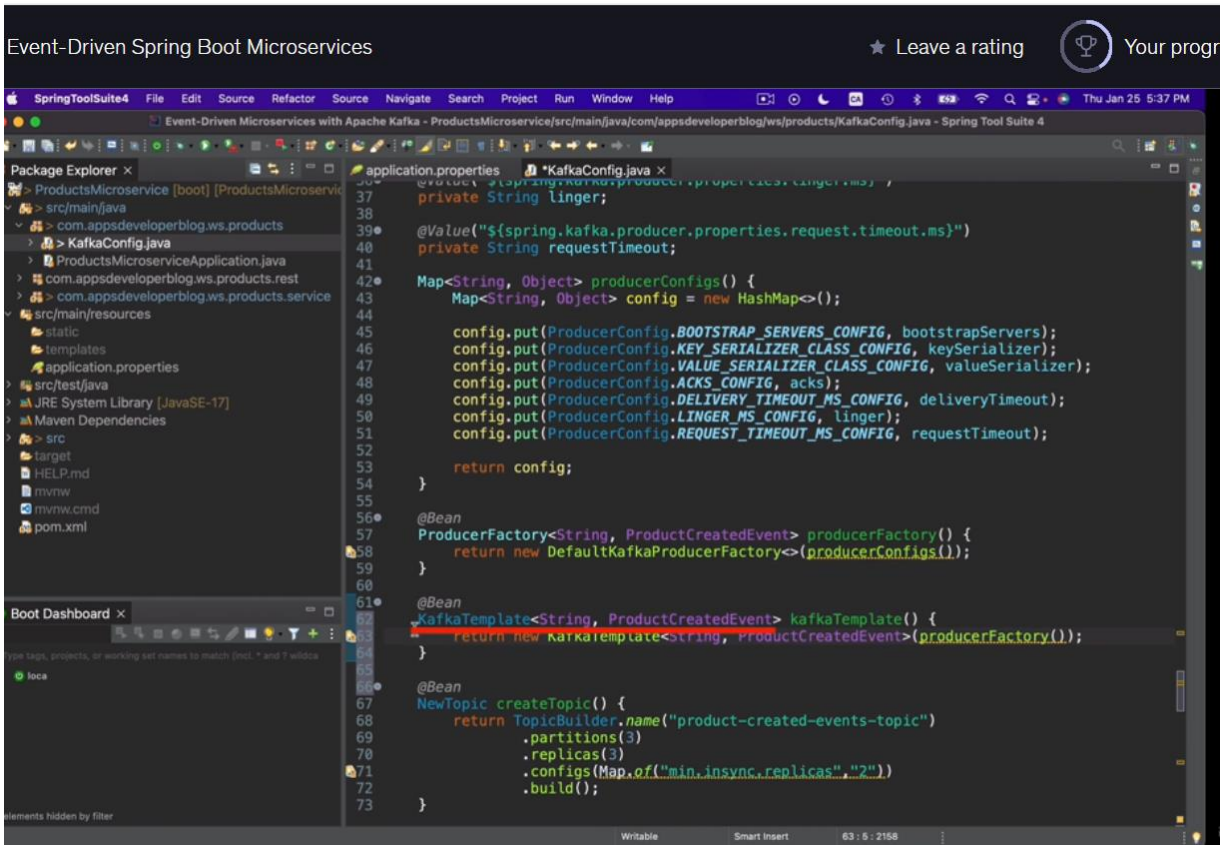
What is the impact of setting 'acks=all' in a Kafka Producer configuration?

What does the 'acks' configuration in a Kafka Producer specify?

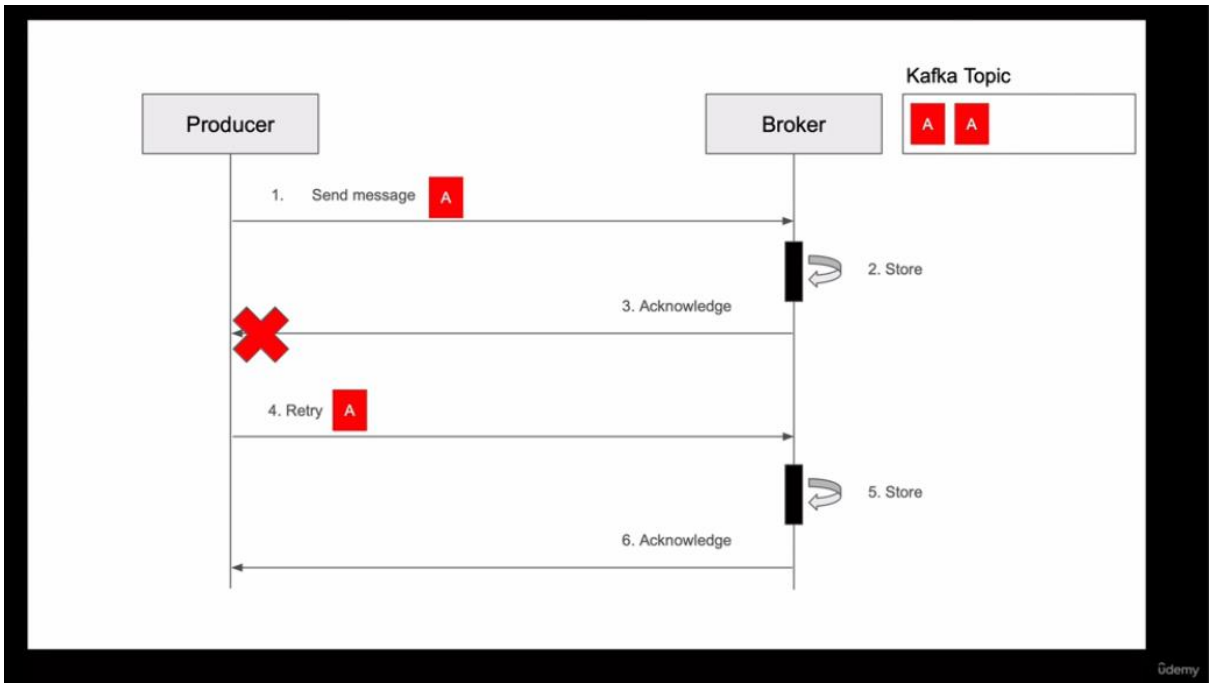
How does the 'retry.backoff.ms' setting affect Kafka Producer retries?

What does the 'retries' configuration in a Kafka Producer control?

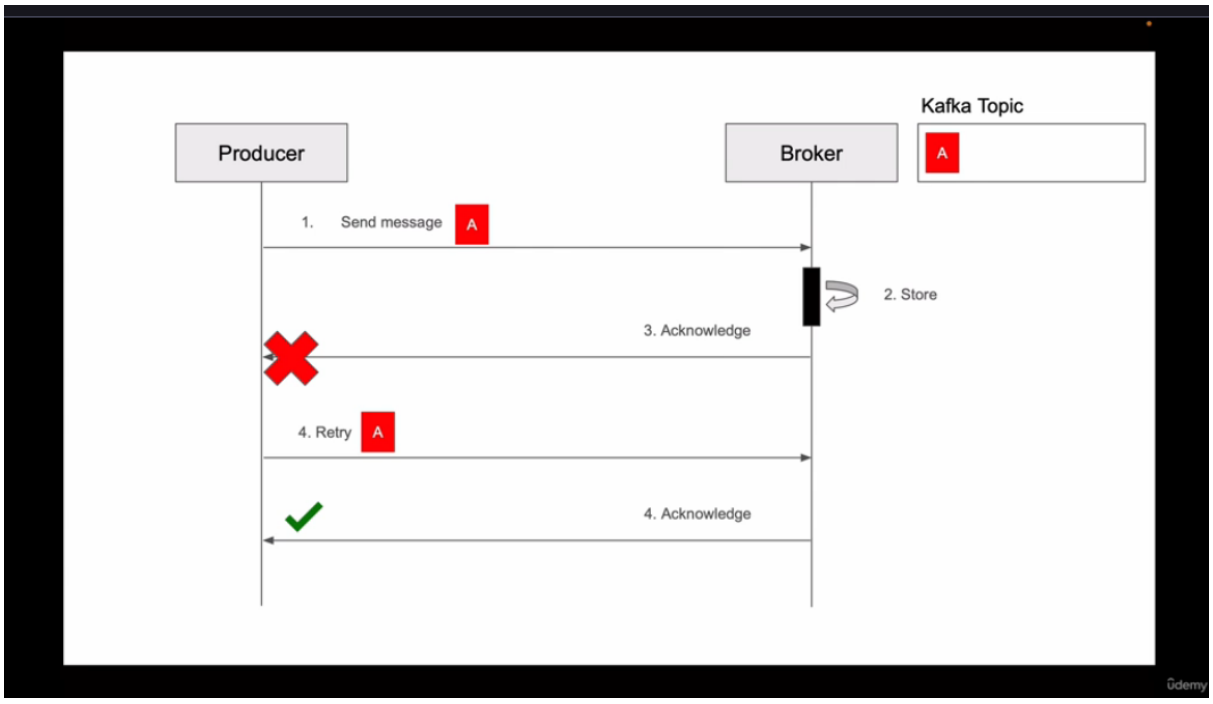
KAFKA PRODUCER USING JAVA BEAN



IDEMPOTENT IN KAFKA:



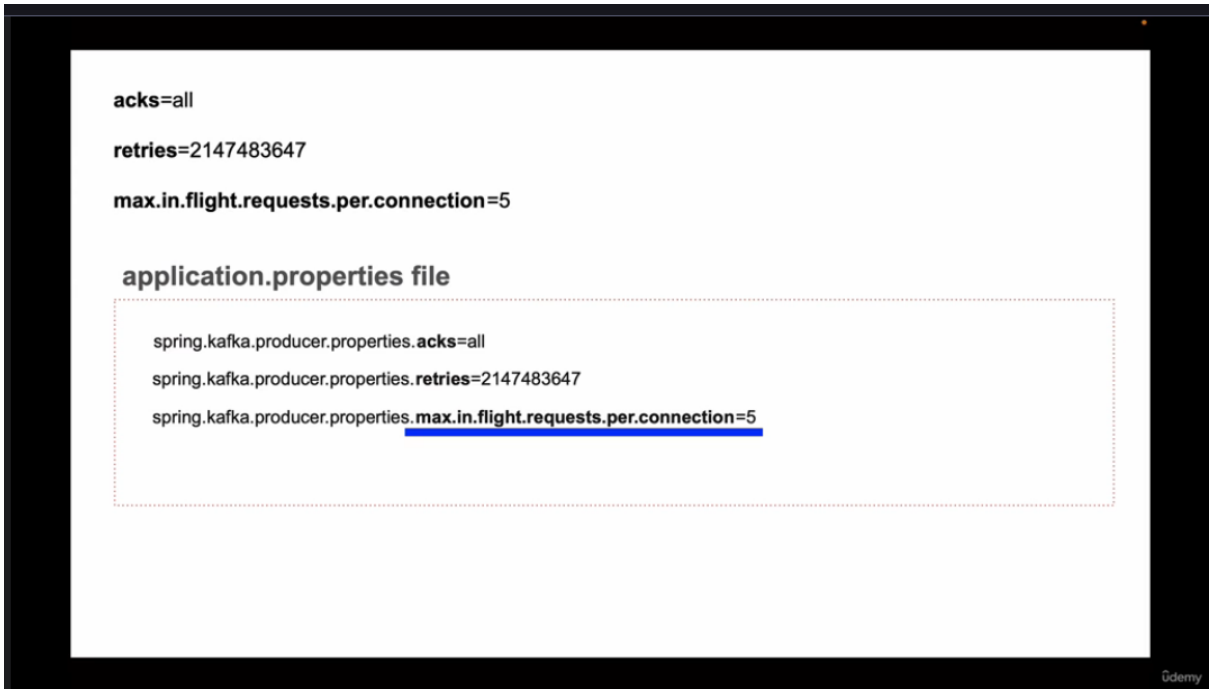
An idempotent producer avoids duplicate messages in the log in the presence of failures and retries.

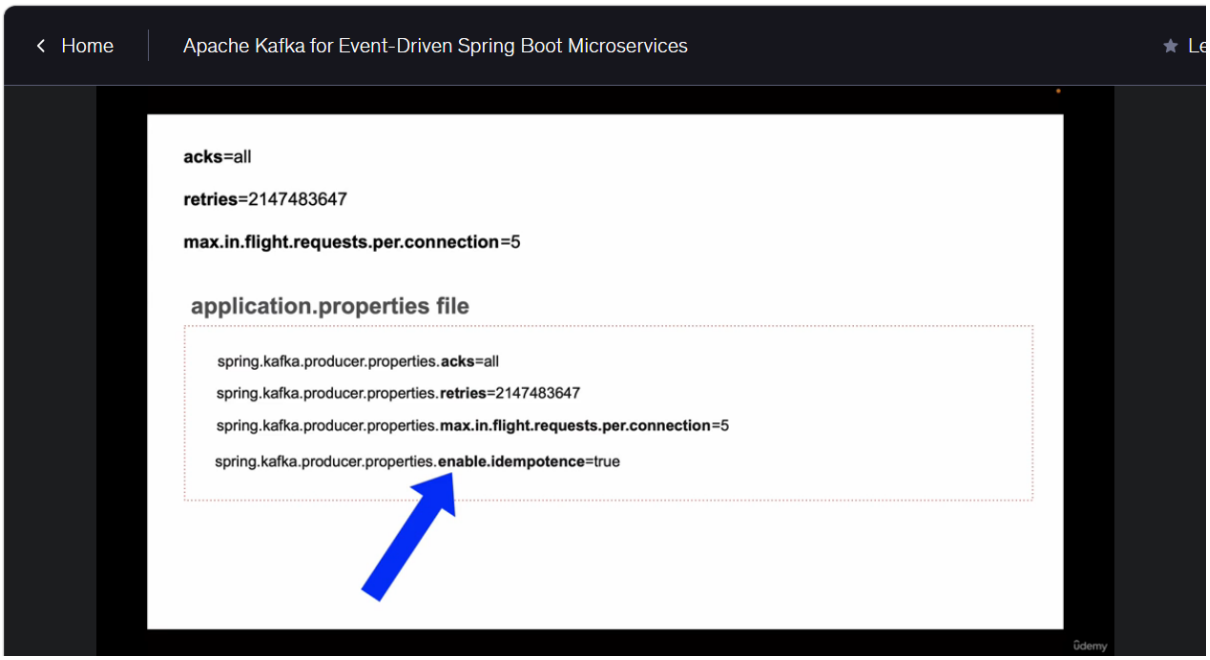


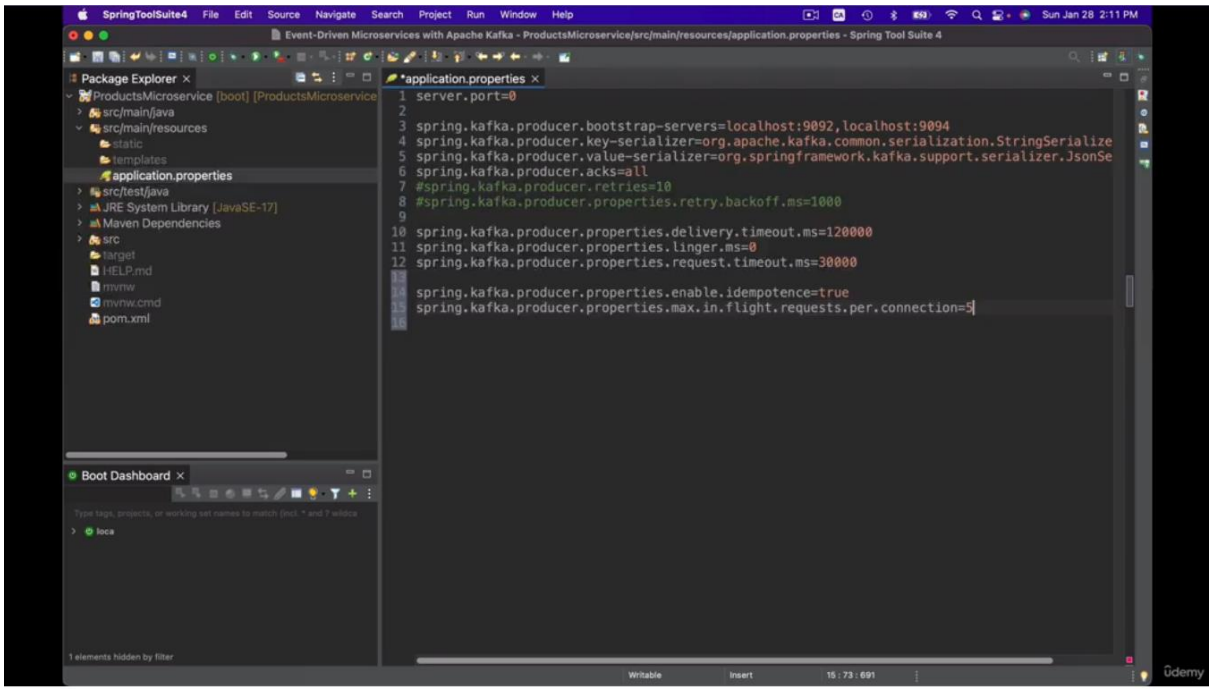
ENABLE IDOMPETENCY:

By default it is enabled



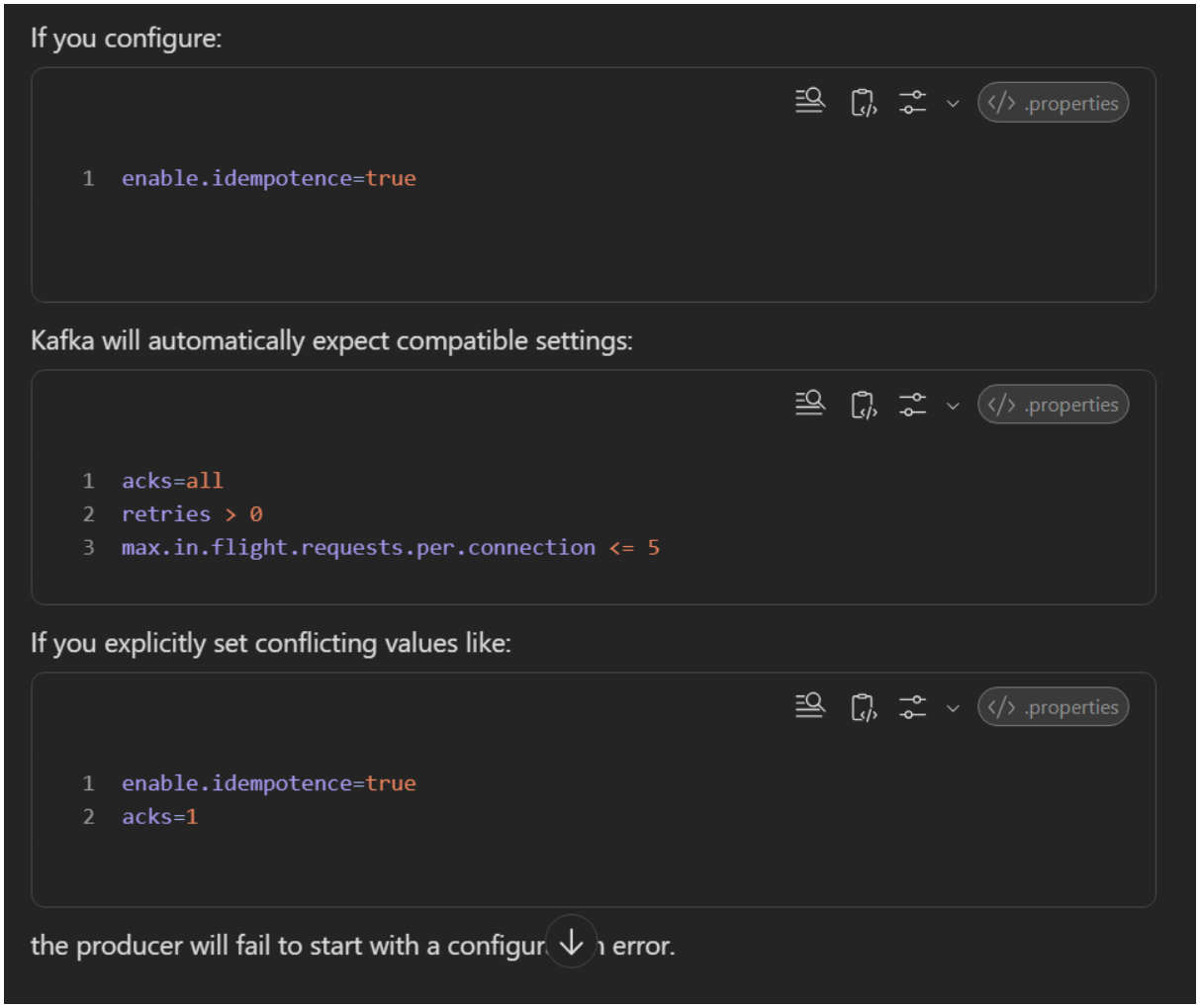




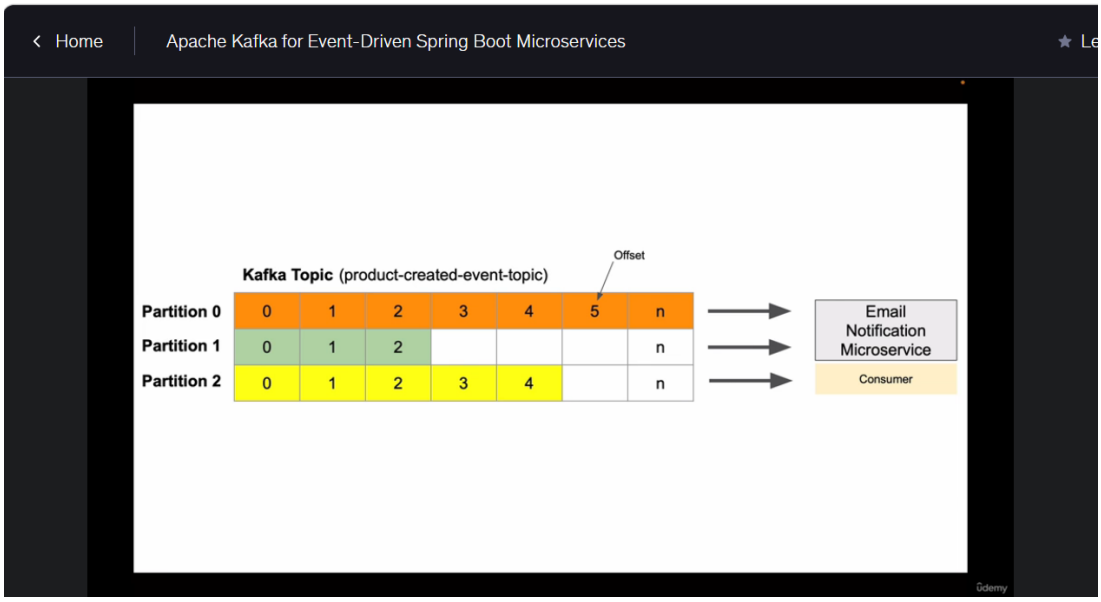


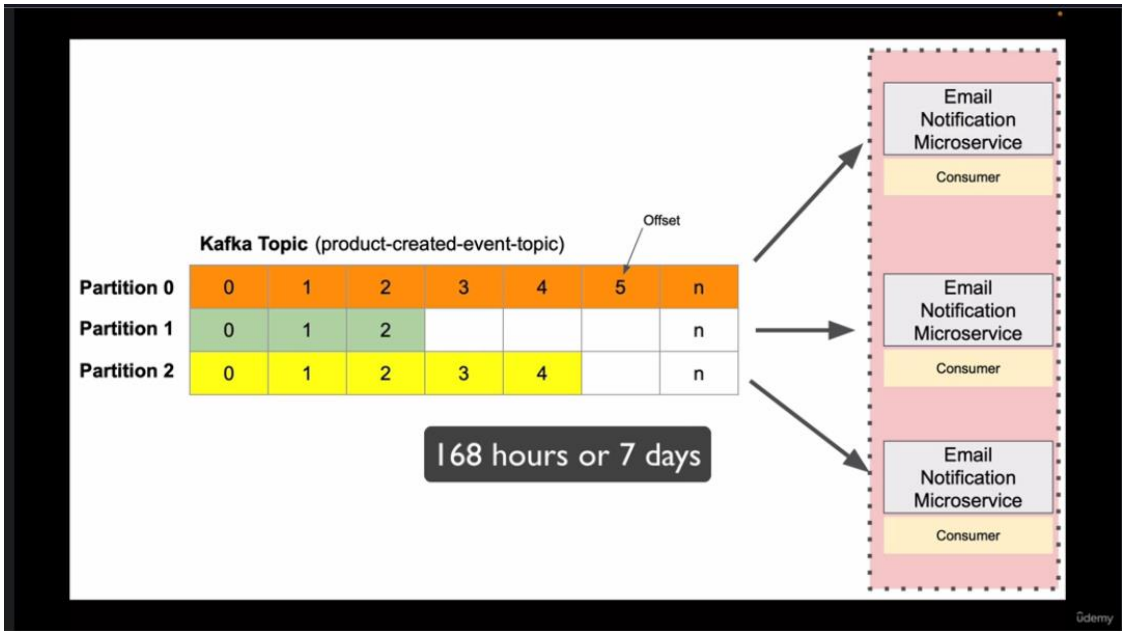
Idempotent producer requires acks=all because Kafka needs a strong acknowledgment from all in-sync replicas before confirming a write. This

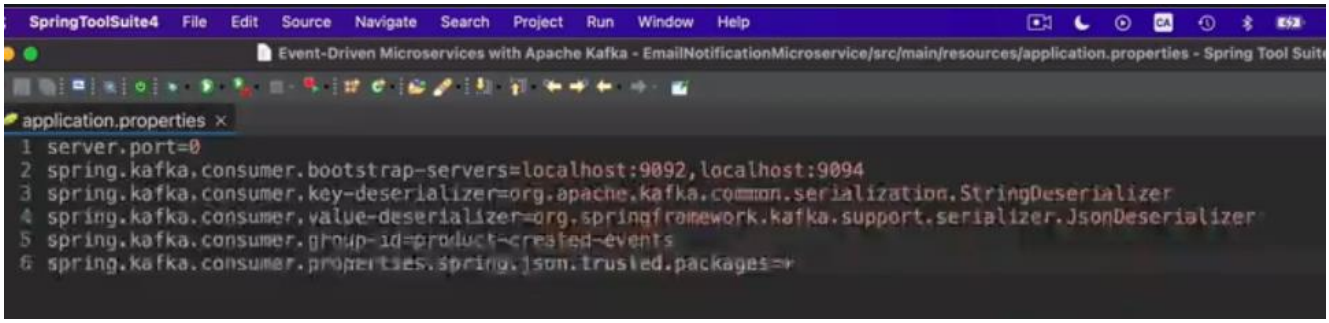
allows retries to be performed safely without duplicates or weakening durability guarantees.



KAFKA CONSUMER:


Home
Apache Kafka for Event-Driven Spring Boot Microservices
Kafka Topic (product-created-event-topic)
Offset
Partition 0
Partition 1
Partition 2
0 1 2 3 4 5 n
0 1 2 n
0 1 2 3 4 n
Email Notification Microservice
Consumer


Kafka Topic (product-created-event-topic)
Offset
Partition 0
Partition 1
Partition 2
0 1 2 3 4 5 n
0 1 2 n
0 1 2 3 4 n
168 hours or 7 days
Email Notification Microservice
Consumer
Email Notification Microservice
Consumer
Email Notification Microservice
Consumer


SpringToolSuite4 File Edit Source Navigate Search Project Run Window Help
Event-Driven Microservices with Apache Kafka - EmailNotificationMicroservice/src/main/resources/application.properties - Spring Tool Suite
application.properties
server.port=0
spring.kafka.consumer.bootstrap-servers=localhost:9092,localhost:9094
spring.kafka.consumer.key-deserializer=org.apache.kafka.common.serialization.StringDeserializer
spring.kafka.consumer.value-deserializer=org.springframework.kafka.support.serializer.JsonDeserializer
spring.kafka.consumer.group-id=product-created-events
spring.kafka.consumer.properties.spring.json.trusted.packages=*

```java
@KafkaListener(
    id = "listener1",      // Spring listener/container id
    groupId = "orders-g1", // Kafka consumer group
    topics = "orders"
)
```

- **id** → Spring identifies the listener.
- **groupId** → Kafka identifies the consumer group.

Example:

```
orders-topic
    -> OrderCreatedEvent
    -> OrderCancelledEvent
    -> OrderUpdatedEvent
```

Then:

```java
@KafkaListener(topics = "orders-topic")
public class OrderConsumer {

    @KafkaHandler
    public void created(OrderCreatedEvent event) {
    }

    @KafkaHandler
    public void updated(OrderUpdatedEvent event) {
    }

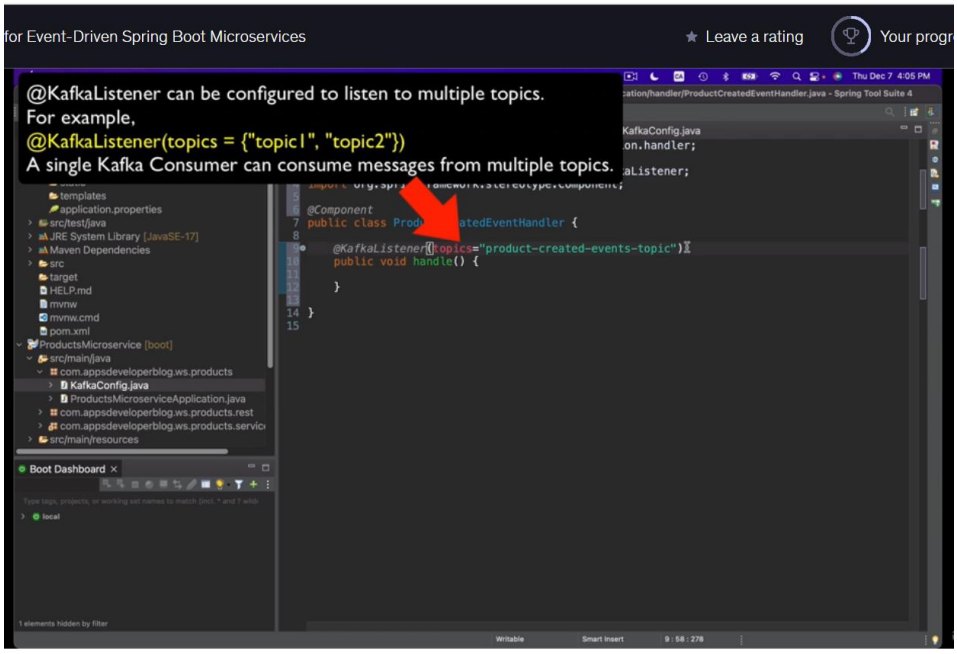
    @KafkaHandler
    public void cancelled(OrderCancelledEvent event) {
    }
}
```

When do we use method-level?

Most projects use this.

```java
@KafkaListener(topics = "orders")
public void consume(OrderCreatedEvent event) {
}
```

Simple. One listener → One message type.



CONSUMER CAN COMSUME MULTIPLE PARITTIONS OR TOPICS

Suppose you have these topics:

```
orders-topic      -> 3 partitions
payments-topic    -> 2 partitions
```

Total partitions:

```
orders-P0
orders-P1
orders-P2

payments-P0
payments-P1
```

One consumer subscribing to both topics

```java
@KafkaListener(
    topics = {"orders-topic", "payments-topic"},
    groupId = "commerce-group"
)
public void consume(String message) {
    // process
}
```

Kafka may assign:

```
Consumer-1
  ├─ orders-P0
  ├─ orders-P1
  ├─ orders-P2
  ├─ payments-P0
  └─ payments-P1
```

The consumer keeps polling from all assigned partitions.

Think of it as:

```
Consumer-1
    |
    +--> orders-P0 messages
    +--> orders-P1 messages
    +--> orders-P2 messages
    +--> payments-P0 messages
    +--> payments-P1 messages
```

SAMPLE JAVA CODE

@Service

@Slf4j

public class CommerceConsumer {

@KafkaListener(

topics = {"orders-topic", "payments-topic"},

groupId = "commerce-group")

public void consume(String message,@Header(KafkaHeaders.RECEIVED_TOPIC) String topic) {

```
switch (topic) {

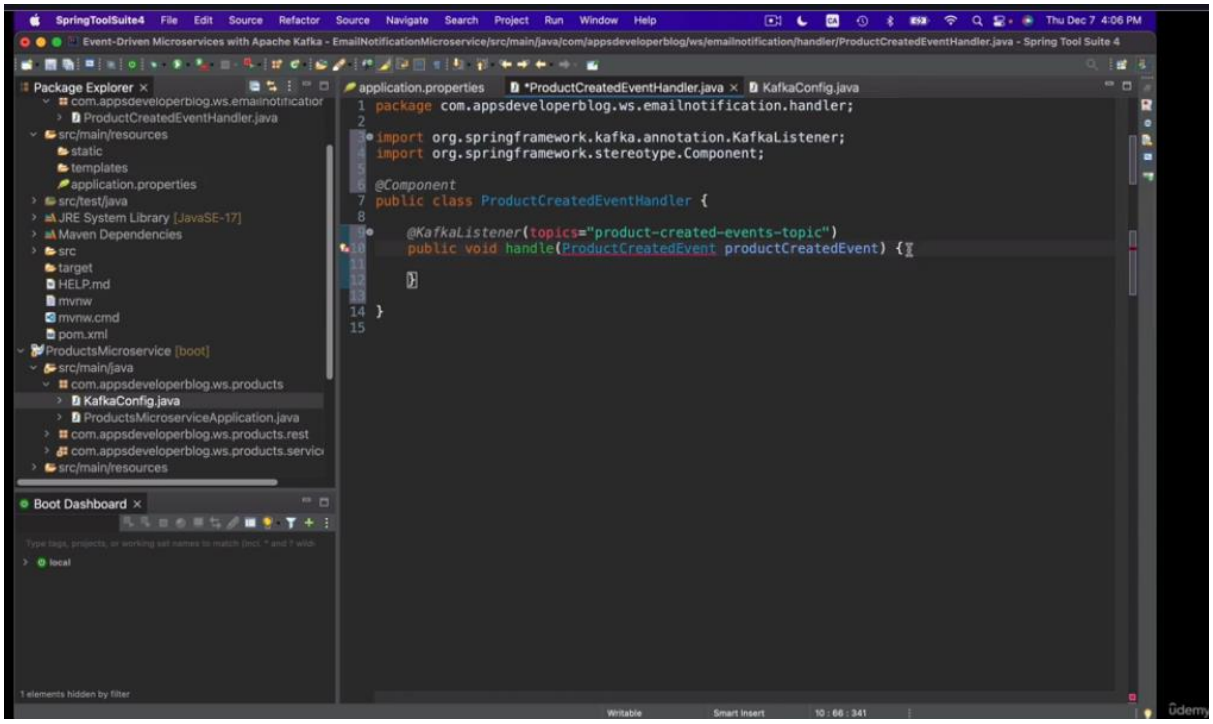
case "orders-topic":
processOrder(message);
break;

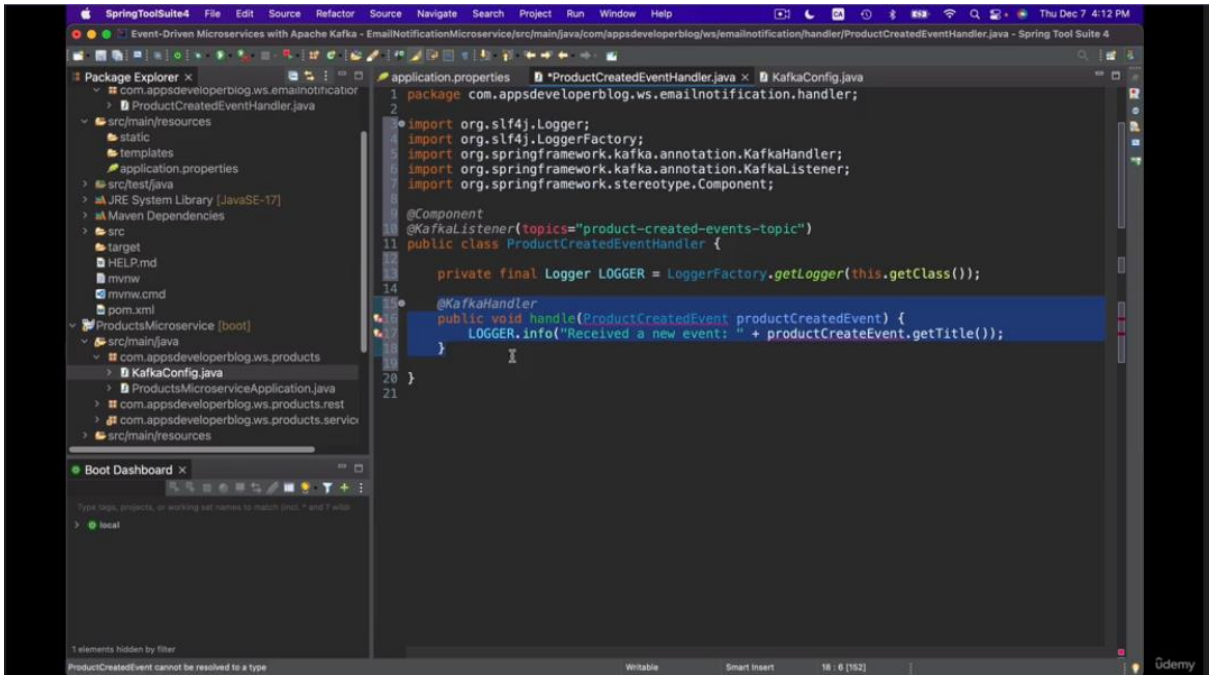
case "payments-topic":
processPayment(message);
break;

default:
log.warn("Unknown topic {}", topic);
}
}

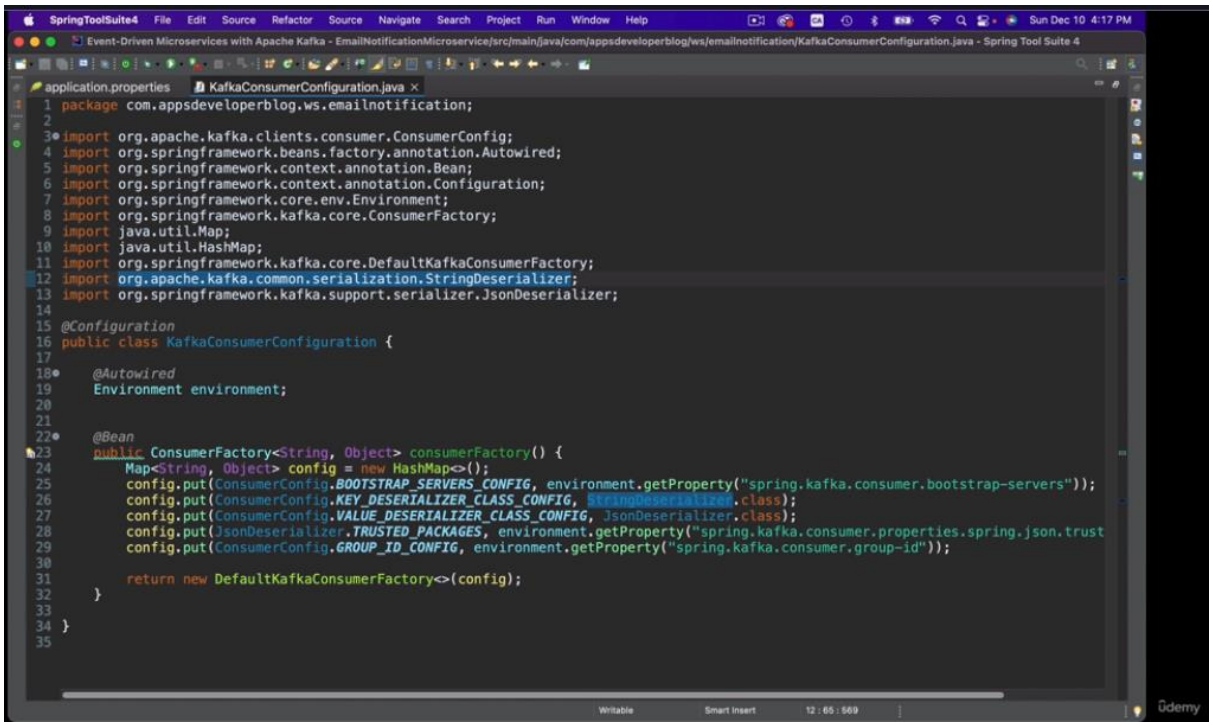
private void processOrder(String message) {
log.info("Processing order {}", message);
}

private void processPayment(String message) {
log.info("Processing payment {}", message);
}
}
```

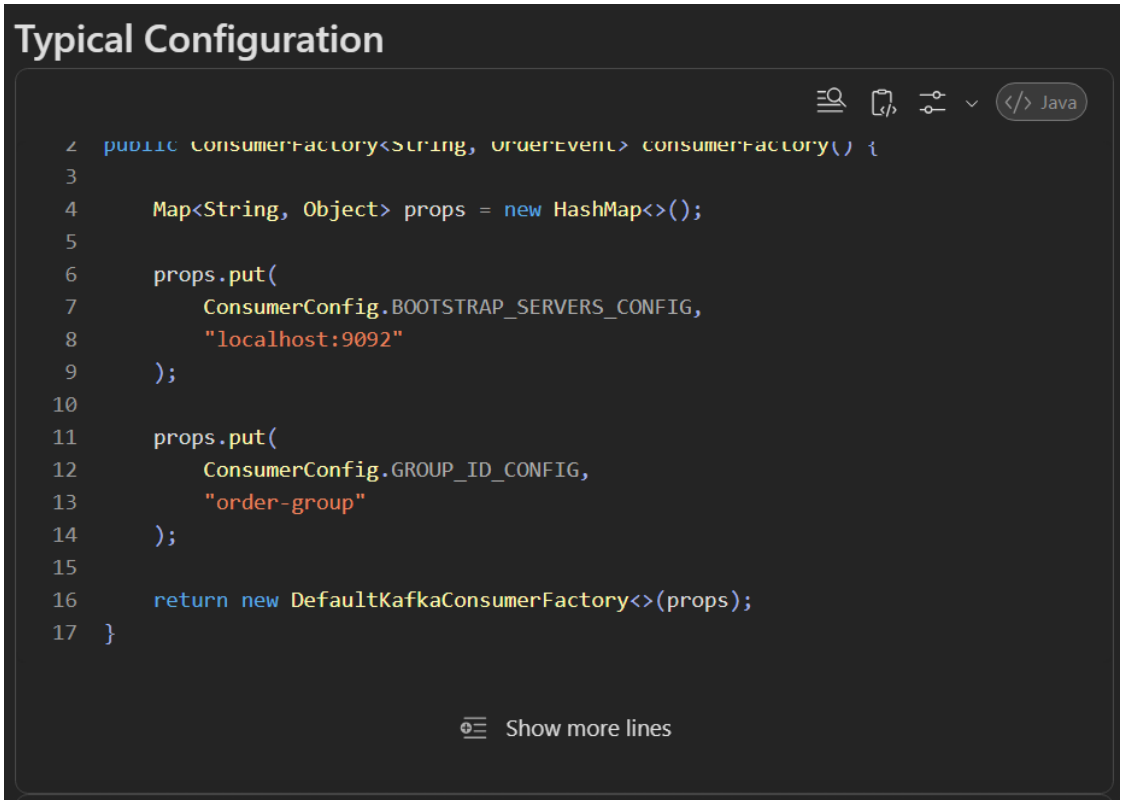

SpringToolSuite4 File Edit Source Refactor Source Navigate Search Project Run Window Help
Thu Dec 7 4:06 PM
Event-Driven Microservices with Apache Kafka - EmailNotificationMicroservice/src/main/java/com/appsdeveloperblog/ws/emailnotification/handler/ProductCreatedEventHandler.java - Spring Tool Suite 4
Package Explorer
com.appsdeveloperblog.ws.emailnotificatior
ProductCreatedEventHandler.java
src/main/resources
static
templates
application.properties
src/test/java
JRE System Library [JavaSE-17]
Maven Dependencies
src
target
HELP.md
mvnw
mvnw.cmd
pom.xml
ProductsMicroservice [boot]
src/main/java
com.appsdeveloperblog.ws.products
KafkaConfig.java
ProductsMicroserviceApplication.java
com.appsdeveloperblog.ws.products.rest
com.appsdeveloperblog.ws.products.servic
src/main/resources
Boot Dashboard
local
1 elements hidden by filter
application.properties
*ProductCreatedEventHandler.java
KafkaConfig.java
1 package com.appsdeveloperblog.ws.emailnotification.handler;
2
3 import org.springframework.kafka.annotation.KafkaListener;
4 import org.springframework.stereotype.Component;
5
6 @Component
7 public class ProductCreatedEventHandler {
8
9 @KafkaListener(topics="product-created-events-topic")
10 public void handle(ProductCreatedEvent productCreatedEvent) {
11
12 }
13
14 }
15
Writable
Smart Insert
10 : 66 : 341
Udemy


SpringToolSuite4 File Edit Source Refactor Source Navigate Search Project Run Window Help
Thu Dec 7 4:12 PM
Event-Driven Microservices with Apache Kafka - EmailNotificationMicroservice/src/main/java/com/appsdeveloperblog/ws/emailnotification/handler/ProductCreatedEventHandler.java - Spring Tool Suite 4
Package Explorer
com.appsdeveloperblog.ws.emailnotificatior
ProductCreatedEventHandler.java
src/main/resources
static
templates
application.properties
src/test/java
JRE System Library [JavaSE-17]
Maven Dependencies
src
target
HELP.md
mvnw
mvnw.cmd
pom.xml
ProductsMicroservice [boot]
src/main/java
com.appsdeveloperblog.ws.products
KafkaConfig.java
ProductsMicroserviceApplication.java
com.appsdeveloperblog.ws.products.rest
com.appsdeveloperblog.ws.products.servic
src/main/resources
Boot Dashboard
local
1 elements hidden by filter
application.properties
*ProductCreatedEventHandler.java
KafkaConfig.java
1 package com.appsdeveloperblog.ws.emailnotification.handler;
2
3 import org.slf4j.Logger;
4 import org.slf4j.LoggerFactory;
5 import org.springframework.kafka.annotation.KafkaHandler;
6 import org.springframework.kafka.annotation.KafkaListener;
7 import org.springframework.stereotype.Component;
8
9 @Component
10 @KafkaListener(topics="product-created-events-topic")
11 public class ProductCreatedEventHandler {
12
13 private final Logger LOGGER = LoggerFactory.getLogger(this.getClass());
14
15 @KafkaHandler
16 public void handle(ProductCreatedEvent productCreatedEvent) {
17 LOGGER.info("Received a new event: " + productCreateEvent.getTitle());
18 }
19
20 }
21
ProductCreatedEvent cannot be resolved to a type
Writable
Smart Insert
18 : 6 [152]
Udemy

SO IN CONSUMER WE HAVE THE SERIALIZER AND DESERIALIZER PACKAGE NOT RECOMMENDED SO WE CAN MAKE USE OF THE JAVA BEAN TO CONFIGURE THAT….



KAFKA LISTENER CONTAINER FACTORY:



```java
@Bean
public ConcurrentKafkaListenerContainerFactory<String, OrderEvent>
        kafkaListenerContainerFactory() {

    ConcurrentKafkaListenerContainerFactory<
        String,
        OrderEvent> factory =
        new ConcurrentKafkaListenerContainerFactory<>();

    factory.setConsumerFactory(consumerFactory());

    return factory;
}
```

Multiple Factories Example

Suppose:

Normal Events

```java
@Bean
public ConcurrentKafkaListenerContainerFactory<String, String>
normalFactory() {
   ...
}
```

Batch Events

```java
@Bean
public ConcurrentKafkaListenerContainerFactory<String, String>
batchFactory() {

    factory.setBatchListener(true);

    return factory;
}
```

Use:

```java
@KafkaListener(
    topics = "orders",
    containerFactory = "normalFactory"
)
public void consume(String msg) {
}
```

```java
@KafkaListener(
    topics = "bulk-orders",
    containerFactory = "batchFactory"
)
public void consume(List<String> msgs) {
}
```

Very Common Real-World Usage: Concurrency

Suppose topic has:

```
orders
 P0
 P1
 P2
```

You can configure:

```java
factory.setConcurrency(3);
```

This creates:

```
Consumer Thread-1
Consumer Thread-2
Consumer Thread-3
```

inside the same application instance

Manual Acknowledgment Example

Normally Kafka commits offsets automatically.

For manual commit:

```java
factory.getContainerProperties()
        .setAckMode(AckMode.MANUAL);
```

Listener:

```java
@KafkaListener(topics = "orders")
public void consume(
        String msg,
        Acknowledgment ack) {

    process(msg);

    ack.acknowledge();
}
```

This is very common in payment/order syste

Error Handling Example

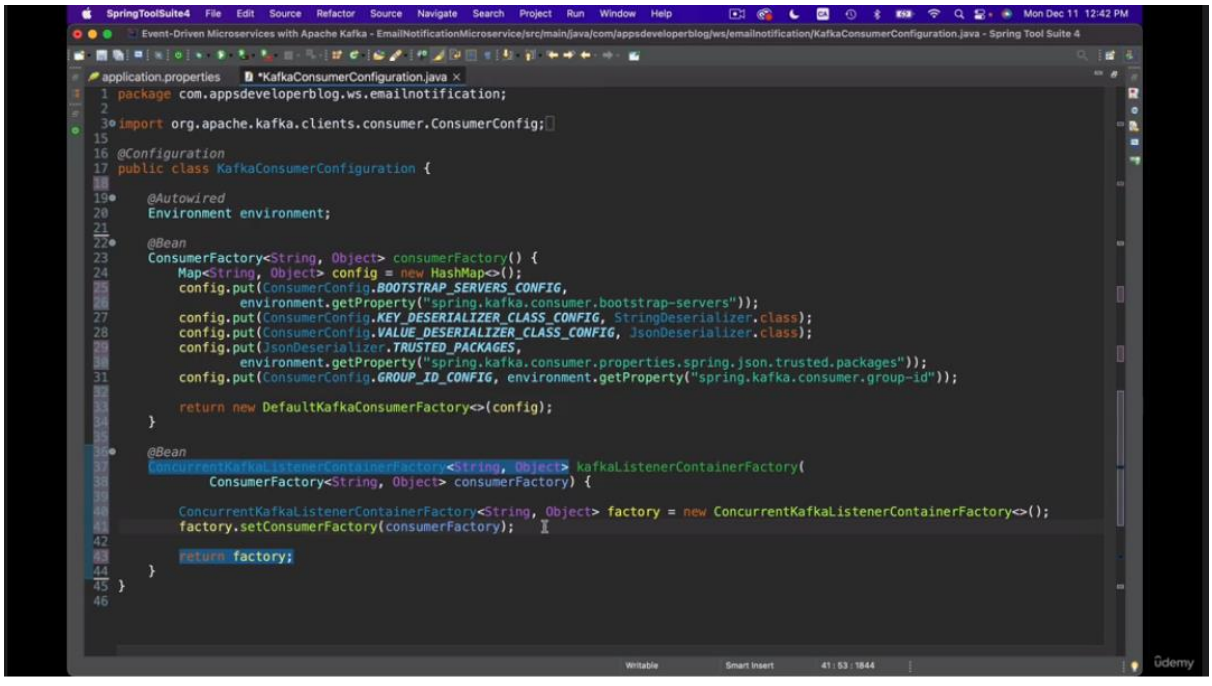
```java
factory.setCommonErrorHandler(
    new DefaultErrorHandler()
);
```

Used for:

- Retries
- Backoff
- Dead Letter Topics (DLT)

Interview Answer

> `KafkaListenerContainerFactory` is a Spring Kafka component responsible for creating and configuring listener containers used by `@KafkaListener`. It is used to define consumer behavior such as concurrency, acknowledgment mode, retries, error handling, deserialization, and batch processing. Every `@KafkaListener` ultimately runs inside a listener container created by this tory.



Yes 🔥. One of the **biggest uses** of `ConcurrentKafkaListenerContainerFactory` in real projects is configuring:

- Deserializers
- Error handling
- Retries
- DLT
- JSON conversion
- Manual ack
- Concurrency

✓ **What you know** ⓘ

Can a single Kafka consumer consume messages from more than one topic?

In a Kafka Consumer application, what role does the `spring.kafka.consumer.prope`

What is the difference between the `spring.kafka.consumer.keydeserializer` anc

What distinguishes the `@KafkaHandler` annotation from the `@KafkaListener` annotat

What is the purpose of the `@KafkaHandler` annotation in a Kafka Consumer applicatio

What is the primary function of a Kafka Consumer?